C2 p2

1. Care este diferența între un microprocesor și un microcontroler?

Principalele diferențe constau în scopul utilizării, structură internă, modul de conectare la resurse și direcția de dezvoltare.

Microprocesorul se poate defini ca un circuit logic programabil de către utilizator, într-o singură capsulă și cu funcție de procesor de uz general. El execută instrucțiuni, procesează date și controlează resursele externe (memorie, I/O) prin magistralele dedicate (de adresă, de date, de control).
Ca particularități avem: – terminalele ocupă un spațiu mare
– nr. mare de porturi
Direcția de dezvoltare: μp tind spre creșterea vitezei și a capacității de memorie adresabile.

Microcontrolerul se poate defini și el ca un circuit logic programabil de către utilizator, cu o structură adaptată pentru rezolvarea unei largi game de aplicații în timp real. Diferența este că el nu este folosit ca un procesor de uz general, ci ca procesor orientat pe aplicații, fiind, în general, inclus în echipamentul pe care îl conduce. El înglobează atât periferice (interfețe seriale, contoare, temporizatoare) cât și porturi care pot fi comandate la nivel de bang ⇒ nr. porturilor externe este mic sau nul.
Direcția de dezvoltare: μc au fost optimizate pt. achiziții de date și aplicații în timp real: electrocasnice, echipamente medicale, etc.

C2

2. Care este diferența între registrele interne ale unui microprocesor și memoria operativă?

Registrele interne ale μP

localizare și acces → sunt o memorie foarte rapidă integrată în procesor

scop → sunt utilizate pentru operații rapide, fiind în strânsă legătură cu setul de instrucțiuni (sunt preferate pt. a stoca operanzi și rezultate intermediare, deoarece memoria externă este mai lentă)

capacitate → au o capacitate limitată de arhitectura μP

Memoria operativă

localizare și acces → e o resursă externă, mai lentă, cu timp de acces mai mare

scop → este utilizată pentru a stoca temporar programele și datele necesare procesării

capacitate → poate avea o capacitate mai mare (de ex RAM de 1 Mo de la 80%)

μP trebuie să comunice cu memoria operativă prin magistrale pt. a o accesa

=> registrele interne pot fi văzute ca o memorie operativă de dimensiune și capacitate redusă, dar de viteză mare

3. Descrieți execuția de către microprocesor a unei instrucțiuni cu operanzi și rezultat.

Execuția unei instrucțiuni este un proces secvențial pentru μP care va fi nevoit să parcurgă un ciclu al instrucțiunilor, divizat în cicluri mașină, iar fiecare ciclu mașină este divizat în stări (perioadă de tact).

Avem următoarele etape:

1. Instruction Fetch (IF)
   - → μP generează semnale de comandă pentru a-și aduce codul instrucțiunii din memorie
   - → μP citește codul instrucțiunii prin magistrala de adrese
   - → μP stochează temporar codul în registrul instrucțiunii (RI)
2. Instruction decode (ID)
   - → din RI codul va fi decodificat de blocul numit DCI (Decodificatorul codului instrucțiune)
   - → DCI transmite informația către DCC (Dispozitiv de comandă și control)
   - → fiind o instrucțiune cu operanzi, procesorul va trebui să genereze semnale de comandă în plus, pentru a aduce și operanzi; DCC inițiază cicluri de citire de date
3. Execute (EX)
   - → în etapa de execuție, μP folosește UAL (Unitate aritmetică și logică) dacă instrucțiunea implică calcule
4. Memory (MEM)
   - → operanzii pot proveni din
     - registre generale (access rapid)
     - tampoane pentru comunicarea cu magistralele
5. Write back (WB)
   - → la sfârșit, rezultatul pleacă dintr-unul din registrele procesorului, prin magistrala de date internă se duce la magistrala de date externă spre exterior, iar μP trece la următoarea instrucțiune

4. Dacă ați fi proiectantul unui microprocesor ce ați alege : mai multe registre generale sau mai multe registre speciale? De ce?

Alegerea depinde de strategia de optimizare a performanței.

(A.) Mai multe registre generale (AX, BX, CX, DX, SP, BP, SI, DI)
→ ele pot fi accesate direct prin instrucțiuni
AVANTAJ: utilizarea lor minimizează nr. de accese la memoria externă (care consumă timp) => crește viteza de execuție

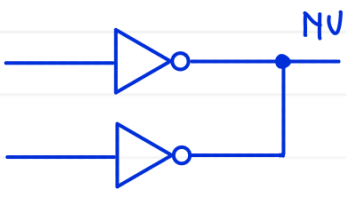
(B.) Mai multe registre speciale (IP, FLAGS)
→ conțin informații dedicate
→ programatorul le poate citi, dar modificarea lor este limitată
AVANTAJ: sunt necesare controlului execuției programelor și implementării facilităților speciale

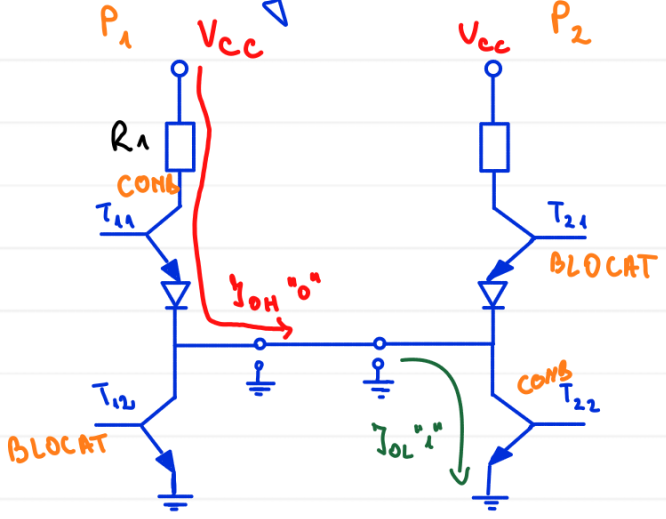
=> Nu există un răspuns unic „corect", vorbim mai degrabă de un compromis care trebuie făcut în ceea ce privește nr. fiecărui tip de registrii, pt. a asigura performanța maximă.

C1

5. Ieșirea unei porți logice normale (cu 2 stări: 0 logic și 1 logic) poate fi legată la o magistrală? Dacă nu, de ce?

Legarea ieșirilor a mai multor circuite care nu au facilitatea de a 3-a stare NU este permisă.



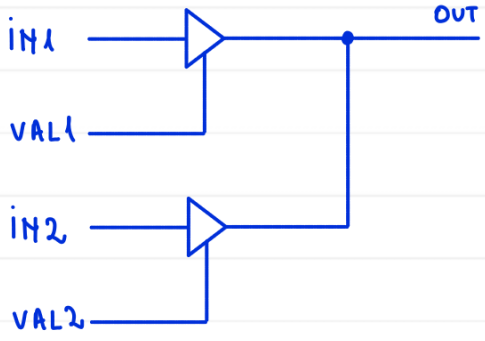

Să presupunem că 2 ieșiri de circuite logice sunt legate împreună, în cazul cel mai defavorabil: unei porți îi cerem „0" logic, iar celeilalte „1" logic.

Avem un conflict, caz în care curentul își găsește o cale de rezistență minimă, prin $T_{11}$ și $T_{22}$ (de la sursă la masă, prin rezistența de 130 Ω)

⇒ curentul care trece este mare, circuitele se încălzesc, iar $R_1$ nu va putea disipa toată căldura ⇒ $R_1$ se arde ⇒ $P_1$ nu mai poate funcționa.

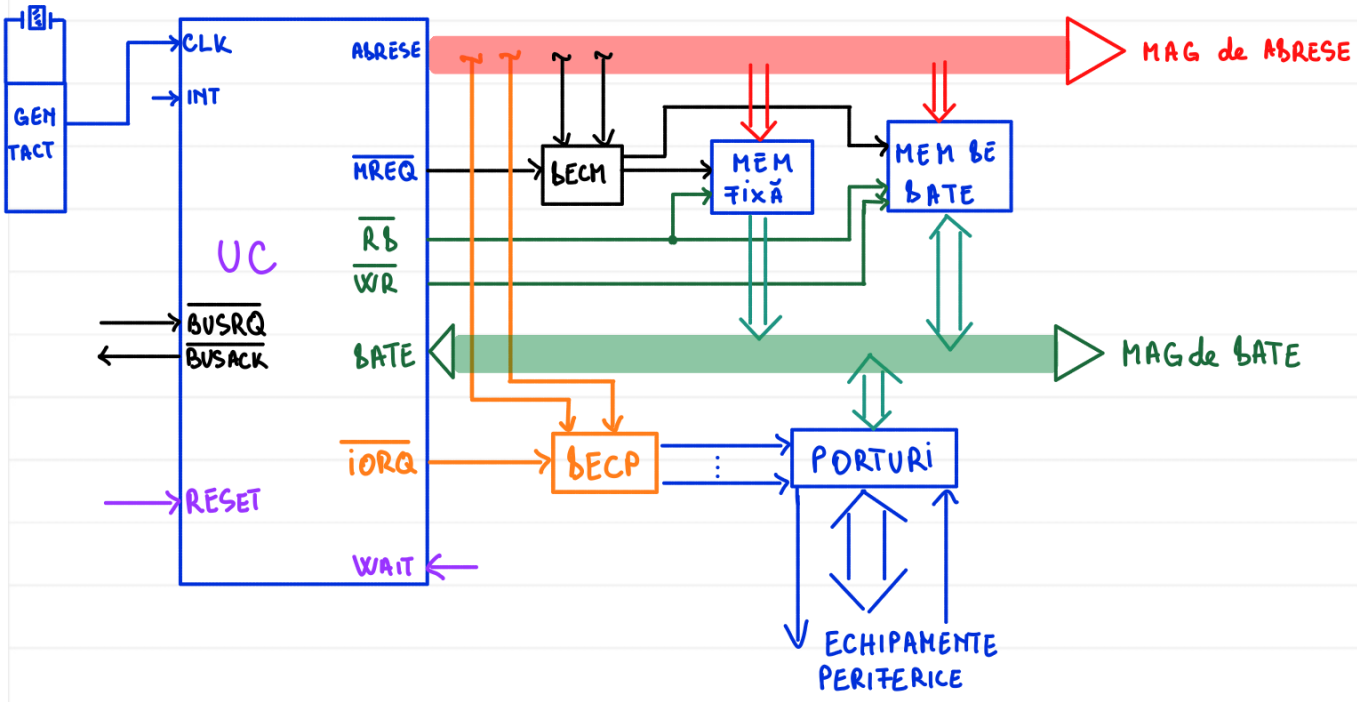
În cazurile în care amlelor porți li se cere aceeași valoare logică, curentul nu poate strica circuitele, deoarece nu își găsește calea de rezistență minimă pentru a avea efect destructiv, iar porțile funcționează normal.

Alternativa este utilizarea porților cu 3 stări, ceea ce ar permite legarea ieșirilor mai multor circuite la o magistrală, cu condiția ca o singură poartă să fie activă la un moment dat.




| VAL 2 | VAL 1 | OUT |
|---|---|---|
| 0 | 0 | a 3-a stare |
| 0 | 1 | IN 1 |
| 1 | 0 | IN 2 |
| 1 | 1 | interzis |

6. Prezentați și comentați schema bloc a unui microsistem digital.




GT → generatorul de tact pe bază de cuarț → trimite semnalul de CLK

UC → unitatea centrală compusă din μP, GT, mecanisme de amplificare și demultiplexare a magistralelor și generatorul de semnal de inițializare.

→ Semnale de control

| | | |
|---|---|---|
| $\overline{RD}$ / $\overline{WR}$ | → | citire / scriere |
| $\overline{MREQ}$ | → | memory request ; procesorul vrea cevа de la mem |
| $\overline{IORQ}$ | → | input/output request ; -II- de la porturi |
| WAIT | → | semnal de așteptare (memoria e lentă) |
| RESET | → | resetarea sistemului |
| INT | → | întreruperi |

DECM → decodificator de memorie: selectează circuitul de memorie pe baza adresei de pe magistrală + MREQ

Avem 2 tipuri de memorii:

Memoria fixă: → ROM, OTP, EPROM, EEPROM sau Flash
→ nu pot fi scrise, doar citite
→ nu își pierde conținutul în cazul în care se pierde tensiunea de alimentare

Memoria de date: → μP poate să și scrie și să și citească aici
→ poți citi de la orice adresă cu aceeași viteză
→ de 2 tipuri | SRAM (Static) → nu are nev. de reîmprospătare
SRAM (Dinamic) → are nev. de reîmprospătare

DECP -> decodificatorul de porturi : -> generează semnale de selecție pentru porturi dacă $\overline{IORQ}$ este activ

-> are ca intrări linii din magistrala de adrese

porturile de intrare ieșire -> asigură interfața dintre UC și echipamentele periferice

-> convertesc informația din formatul UC în cel al periferiecelor și invers

Magistrale externe -> de adrese -> unidirecțională

-> de date -> bidirecțională

4. Câte unități interne are microprocesorul 8086 și care este rolul lor?

Microprocesorul 8086 are 2 unități interne (EU și BIU) care funcționează în paralel: BIU preia informațiile și instrucțiunile, iar EU le execută.

1. EU (Execution unit) → execută
   - → când are nevoie de date, îi cere unității de interfațare cu magistralele
   - → nu are acces la magistrale
   - → include UAL pt. operații matematice și logice
   - → conține registrele generale (AX, BX, CX, DX), registre index/de bază (SP, BP, SI, DI) și registrul de indicatori (FLAGS)
   - → decodifică instrucțiuni de la BIU
2. BIU (Bus interface unit) → duce operanzi, furnizează cod pt EU
   - → asigură leg microprocesorului cu exteriorul
   - → calculează adresele și aduce instr. de memorie în avans, stocându-le într-un șir de instrucțiuni (coadă)
   - → conține o unitate aritmetică pt. calcul de adrese
   - → conține registrele segment (CS, DS, ES, SS)
   - → folosește IP pt. a ține evidența instrucțiunii curente

C2

8. Cum este organizată și gestionată memoria la microprocesorul 8086?

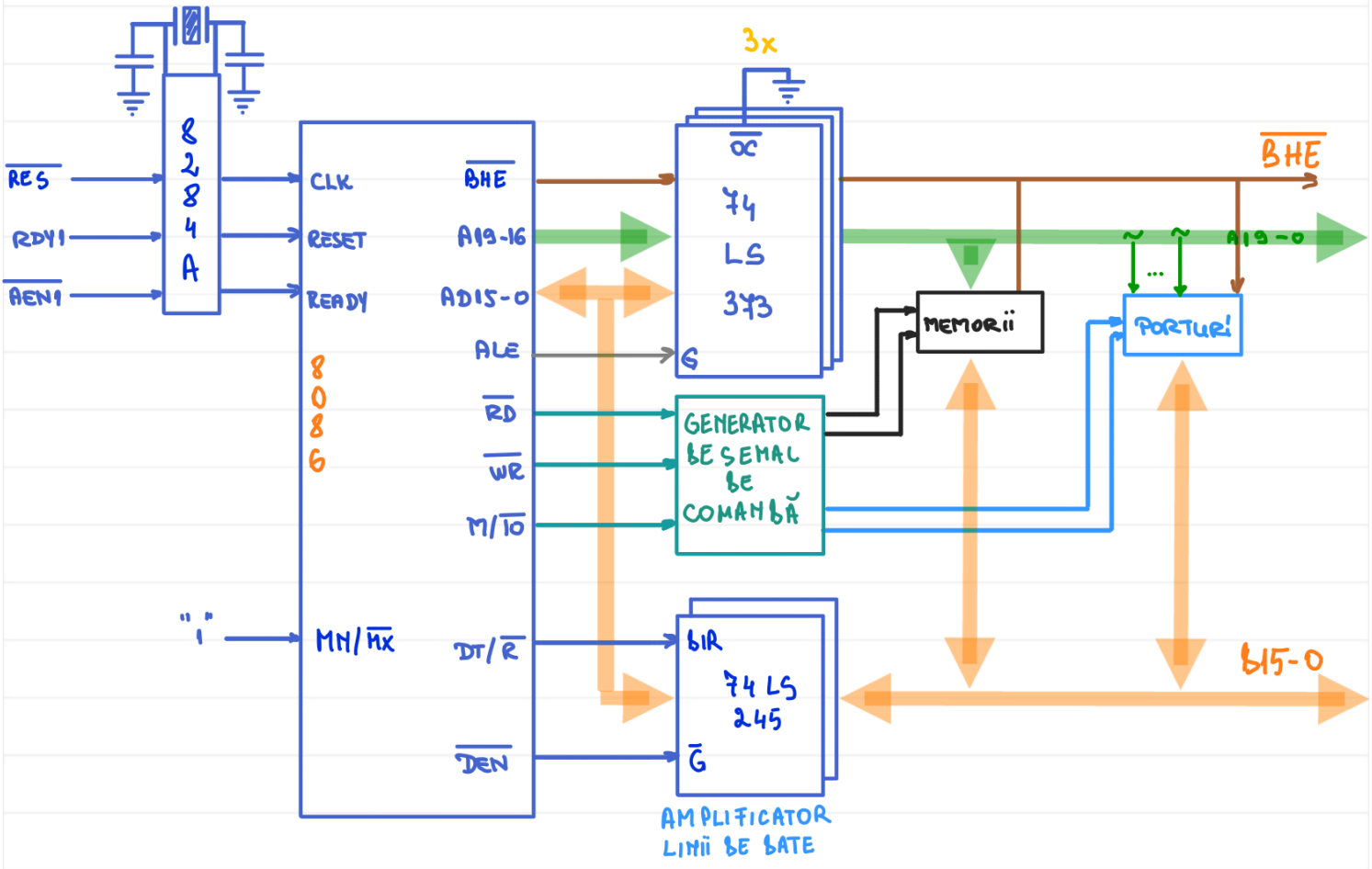
Organizarea și gestionarea memoriei la microprocesorul 8086 poate fi descrisă prin următoarele caracteristici:

1. Capacitatea de memorie direct adresabilă este de 1 MO = $2^{20}$ octeți = 1024 KO
2. Memoria este gestionată în segmente, accesate prin intermediul registrelor de segment (CS, DS, SS, ES)

→ facilitează programarea modulară; fiecare modul poate ocupa unul sau mai multe seg.
→ facilitează relocarea dinamică a programelor; programul poate fi plasat oriunde în mem.
→ se poate utiliza un număr mare de stive (seg stivă SS, vârf stivă SP)
→ limitează lungimea programelor
→ scade viteza de lucru: adresa fizică se obține din 2 configurații

adr. fizică = segment × 16 + deplasament

9. Prezentați schema unității centrale cu microprocesorul 8086 în modul minim.



10. Ce înseamnă existența a mai multor moduri de lucru la un microprocesor? Exemplificați pe un microprocesor.

Existența mai multor moduri de lucru la un microprocesor înseamnă capacitatea acestuia de a funcționa în configurații hardware diferite, adaptându-se la complexitatea sistemului în care este utilizat.

De exemplu, microprocesorul 8086 are 2 moduri de lucru (minim și maxim) selectabile hardware prin pinul MN/$\overline{MX}$

1. MN -> "1" -> este destinat aplicațiilor simple, în care microprocesorul își gestionează el semnalele necesare transferurilor cu memoria și porturile intrare/ieșire.
2. $\overline{MX}$ -> "0" -> este destinat aplicațiilor complexe, unde semnalele de comandă pt porturi și memorii sunt gestionate de un controler de magistrală (de ex 8288).

Aceste moduri nu oferă privilegii diferite, ci se recomandă în anumite configurații hardware, în funcție de tipul de aplicație.

11. Care sunt avantajele modului protejat al gestionării memoriei la microprocesorul 80386?

Modul protejat reprezintă un salt major față de arhitecturile anterioare, oferind următoarele avantaje:

1. performanțe deosebite și suport hardware complet pt multitasking.
asigură comutarea rapidă a taskurilor; fiecare task are un TSS (Task State Segment), iar ele sunt în zone de memorie diferite și protejate pt a evita corupereea task-urilor. Un plus este și separarea și protecția zonelor de memorie proprie a sistemului de operare.

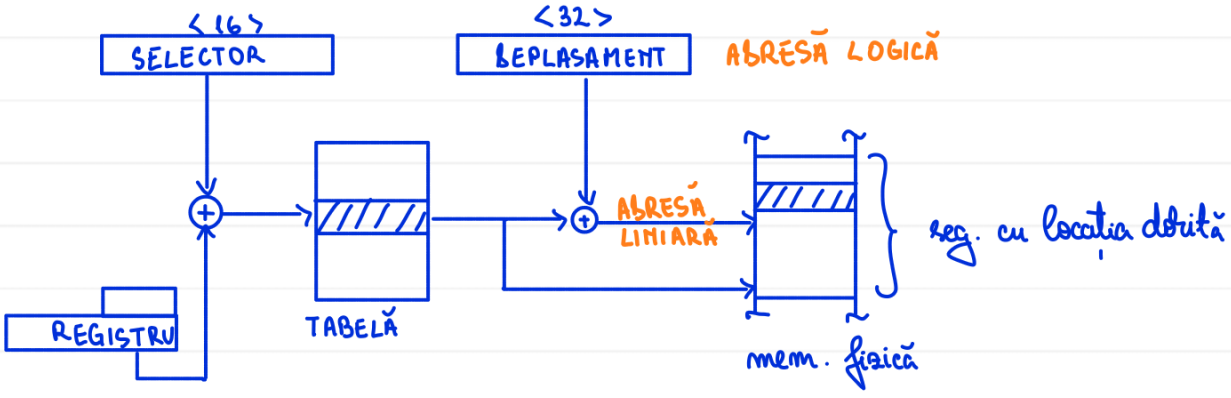
2. are 4 nivele de privilegii: 0 (cel mai privilegiat, corespunde nucleului sistemului)-3 (cel mai puțin privilegiat - aplicații utilizator). Astfel sunt împiedicate accese neautorizate la date sensibile și executarea instr. privilegiate de către programele utilizator.

3. permite fiecărui task să îi fie alocați 64TO memorie virtuală care poate fi mapată în 4 GO mem fizică.

4. Orice acces la memorie este posibil doar prin respectarea unui set de reguli bine determinat care include controale de tip acces, drepturi de acces și nivel privilegiu.

12. Descrieți calea prin care un microprocesor 80386 ajunge la o locație de memorie considerând că mecanismul de paginare nu este activ.

Când mecanismul de paginare este inactiv, µP folosește direct mecanismul de segmentare




Adresa logică are 2 câmpuri:
- selector (aflat într-un registru segment)
- deplasament (offset de 32 de biți)

Selectorul va fi folosit pt a accesa un descriptor din GDT sau LDT
În acest descriptor se află baza segmentului și limita acestuia
Apoi se calculează adresa liniară = baza segmentului + deplasament
Dacă este depășită limita ⇒ se generează o excepție
Din moment ce paginarea nu este activată, adresa liniară devine adresă fizică.

13. Câtă memorie virtuală și câtă memorie fizică poate fi gestionată de un task la μP 80386? Justificați

În mod protejat:

→ 64TO mem virtuală

→ 4 GO mem. fizică

Gestionarea memoriei se face prin mecanismele de segmentare și paginare

1. Segmentarea

Un task are acces la 2 tabele GBT și LBT și poate avea până la $2^{14}$ segmente ($2^{13}$ seg comune + $2^{13}$ seg. proprii)

Descriptorul pt. segmente de cod și date conține rangul de granularitate, adică indică dacă segmentul este văzut ca fiind alcătuit din octeți sau din pagini de 4 KO

Adresa fizică este de 4 GO, din adresarea directă: adrese pe 32 biți $\Rightarrow 2^{32}$ segmente adresabile $\Rightarrow 2^2$ Go = 4GO

Cum un task poate gestiona $2^{14}$ segmente $\Rightarrow 2^{14} \cdot 2^2 \cdot 2^{30} = 2^{46} = 2^6$ To = 64To mem virtuală.

2. Paginarea

Dacă e activă, segmentarea generează adrese liniare, care sunt apoi transformate în adrese fizice de paginare

14. Care este calea prin care un task nou ajunge la segmentele proprii de memorie?

Fiecărui task îi corespunde un segment special TSS (Task State Segment) care conține toate informațiile necesare actualizării stării µP.

TSS conține un câmp care va servi drept selector pentru LDT, prin care noul task va ajunge la segmentele proprii.

Sistemul de operare creează descriptorii de segment în GDT sau LDT și fiecare segment conține baza, lungimea și drepturile de acces.

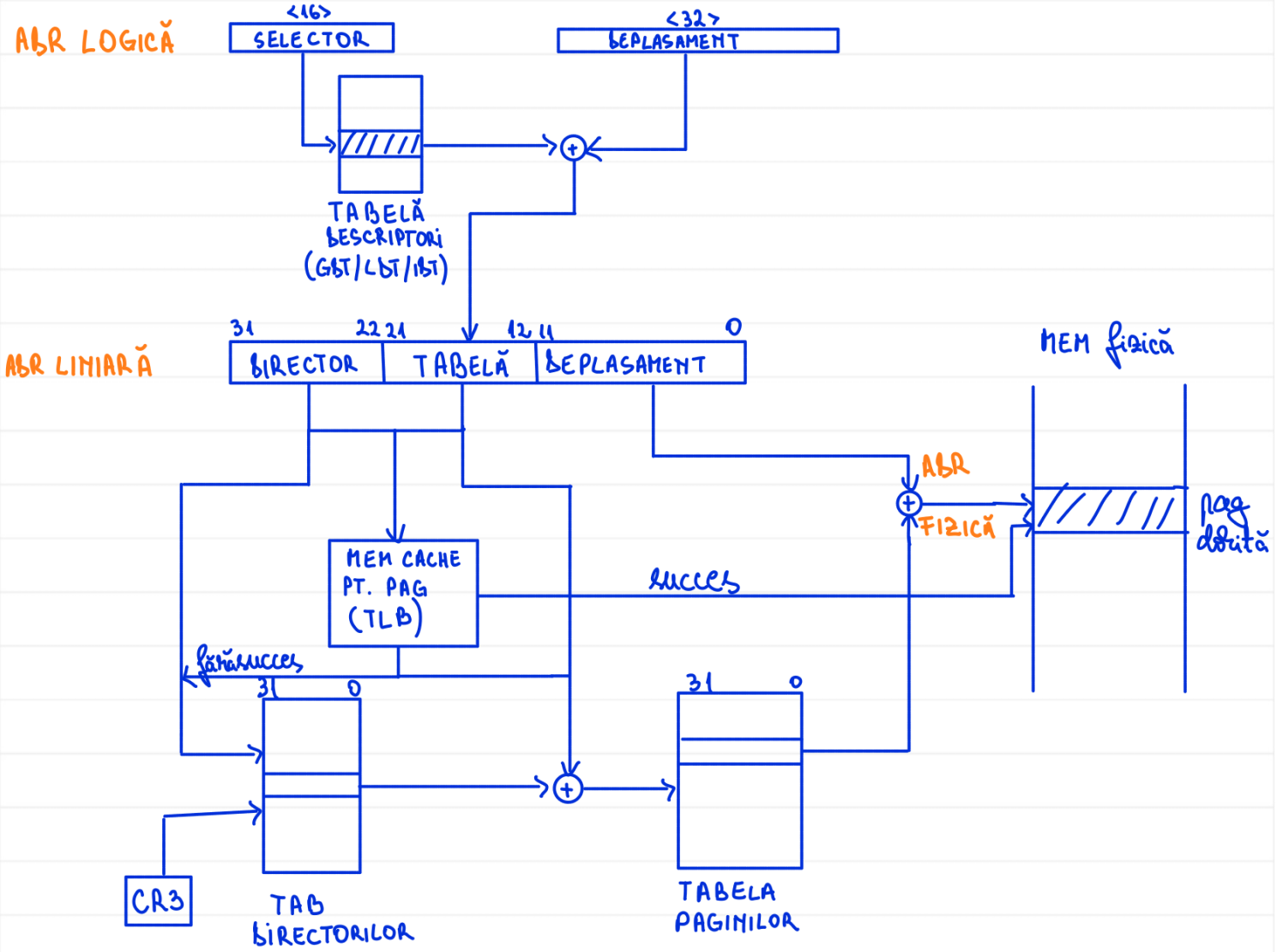
µP preia selectorul LDT și îl încarcă în partea vizibilă a grupului de registre LDTR.

Ulterior, se intră pe mecanismul de segmentare, unde se furnizează adrese de început ale segmentelor locale.

Practic LDTR indică locația tabelei LDT în memorie (cu segmentele de cod și date aferente task-ului), lucru ce asigură izolarea - fiecare task vede și accesează doar seg din propria LDT plus cele comune (GDT)

15. Descrieți calea prin care μP 80386 ajunge la o locație de memorie, considerând că mecanismul de paginare este activ.

Când mecanismul de paginare este activ, μP 80386 accesează o locație de memorie în 2 etape: mecanismul de segmentare furnizează adresa liniară, iar ea este transformată în adresă fizică de mecanismul de paginare.




Adresa logică are 2 câmpuri:
- selector (aflat într-un registru segment)
- deplasament (offset de 32 de biți)

Selectorul va fi folosit pt a accesa un descriptor din GDT sau LDT
În acest descriptor se află baza segmentului și limita acestuia
Apoi se calculează adresa liniară = baza segmentului + deplasament
Dacă este depășită limita ⇒ se generează o excepție

Adresa liniară este împărțită în 3 câmpuri:

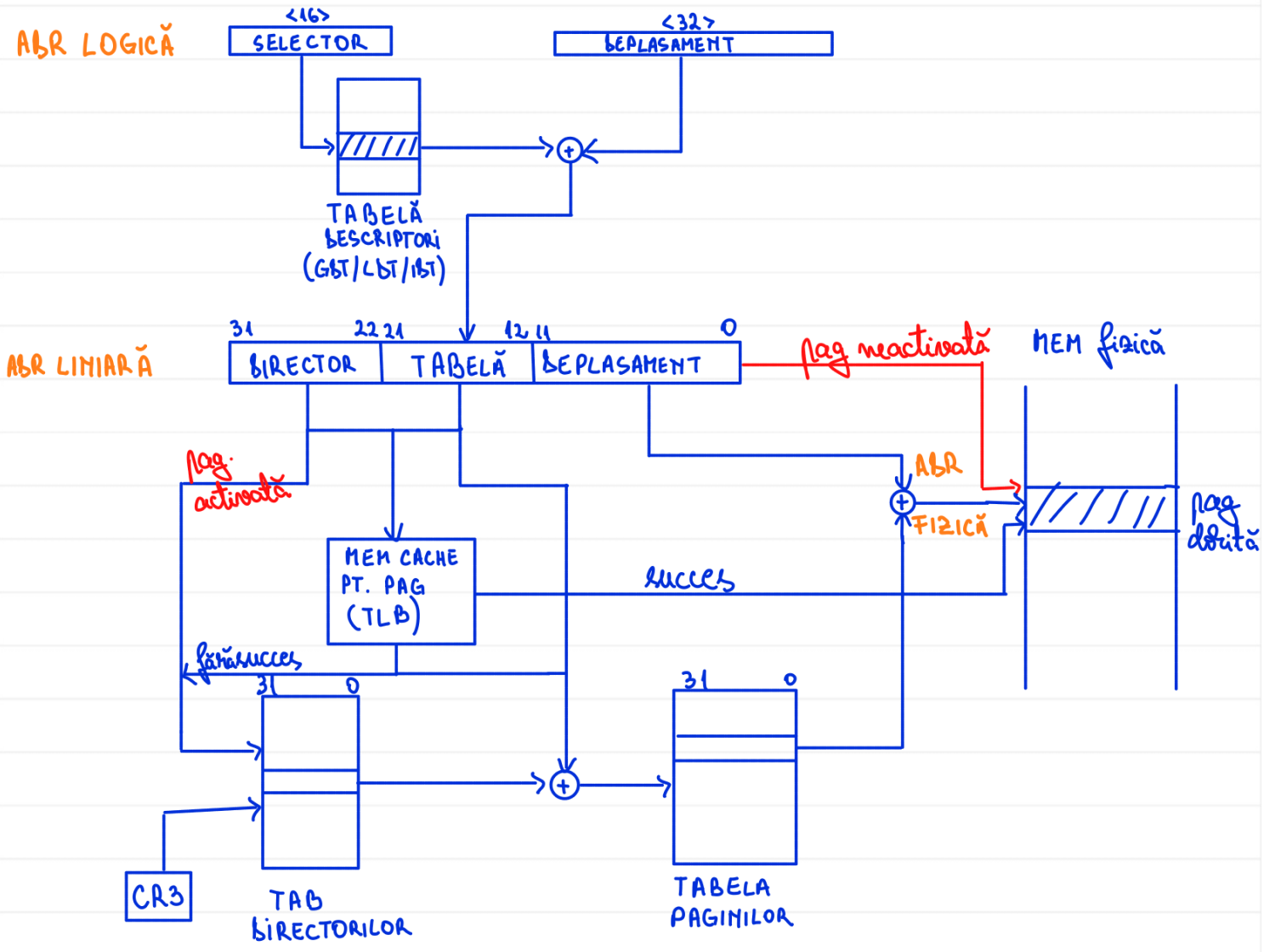
- director -> deplasament în tabela directoriilor
- tabelă -> ne duce la o intrare din tabela paginilor
- deplasament

După obținerea adresei liniare se verifică TLB; dacă intrarea este găsită cu succes, adresa fizică a pagini este obținută imediat.

Dacă nu:

- câmp director + reg. CR3 indică adresa unei tabele de pagini, care adunată cu câmpul tabelă ne va selecta o intrare în tabela de pagini care conține adresa de început a paginii fizice
- se adună câmp deplasament => adresa fizică dorită

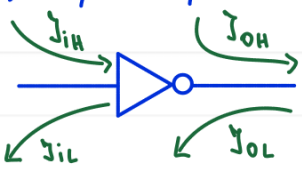
16. Prezentați schema bloc a adresării memoriei la μP 80386.



C4

17. De ce este necesară amplificarea liniilor unui microprocesor? Prezentați soluții pt. linii uni și bidirecționale.

Problema amplificării apare datorită faptului că un circuit logic are pe intrări nu doar tensiuni, ci și curenți.




Pt. a funcționa corect pe o ieșire pot pune maxim $\min\left(\frac{I_{OH}}{I_{IH}}; \frac{I_{OL}}{I_{IL}}\right)$
Dacă nu se respectă cerința, nivelele logice sunt alterate.

⇒ Amplificarea este necesară din cauza limitărilor de putere ale microprocesorului în raport cu sarcina totală a sistemului. µP furnizează curenți mici care ar putea comanda un număr mic de circuite, insuficient pt. sistem.

a) Linii unidirecționale

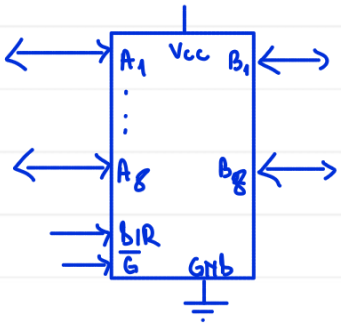
i. Fără facilitatea de a 3-a stare : ex.

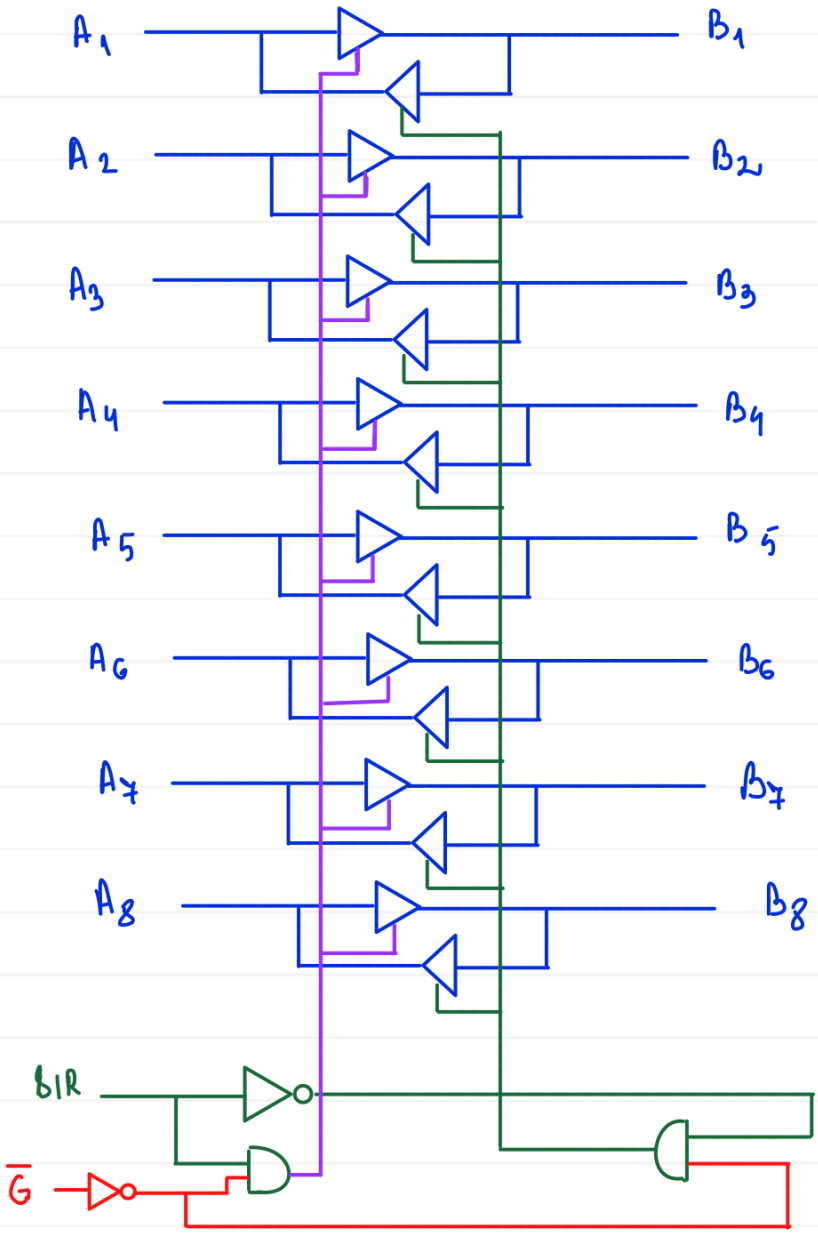

ii. cu facilitatea de a 3-a stare

- soluția optimă, sunt un fel de buffere, pot comuta între stare activă și stare de impedanță, iar semnalul $\overline{BUSACK}$ permite cedarea magistralei

74126 74125

b) Linii bidirecționale



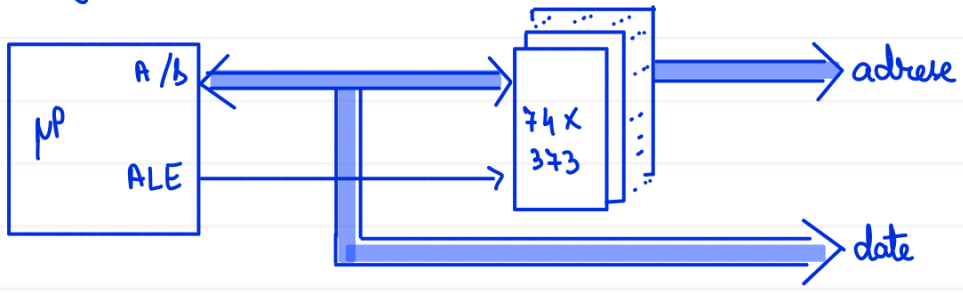

A1
A2
A3
A4
A5
A6
A7
A8
B1
B2
B3
B4
B5
B6
B7
B8
SIR
G

18. Ce înseamnă multiplexarea liniilor unui μP? Este necesară demultiplexarea? Dacă da, de ce și cum?

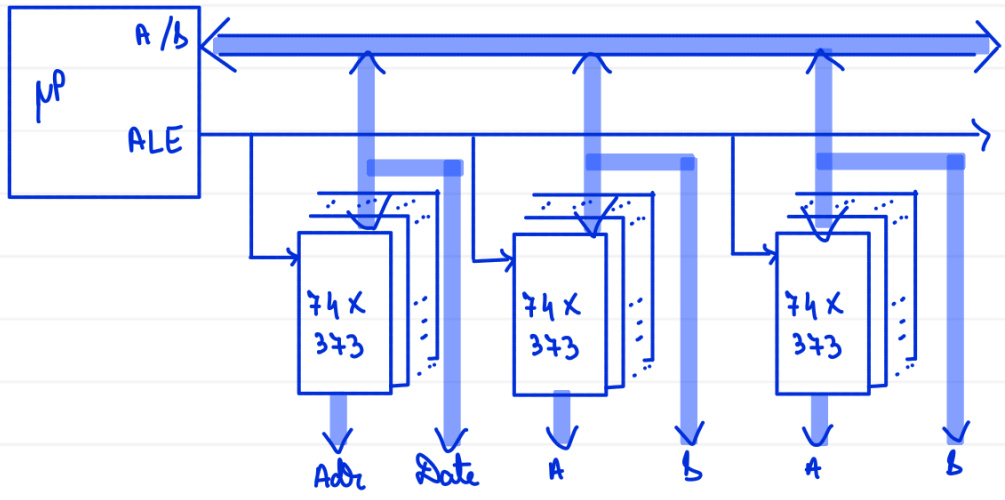
Multiplexarea liniilor implică utilizarea aceleiași linii fizice pentru mai multe tipuri de semnale (adrese și date) la momente de tip diferite
=> utilizarea unui nr. mai mic de pini pe μP => reducerea costului și a dim μP

Demultiplexarea nu ar fi necesară dacă μP ar avea porțenări care lucrează cu linii multiplexate. Memoriile nu lucrează cu linii multiplexate.
Există 2 soluții:

a) generală - dacă se strică o linie de adrese tot sistemul va funcționa eronat

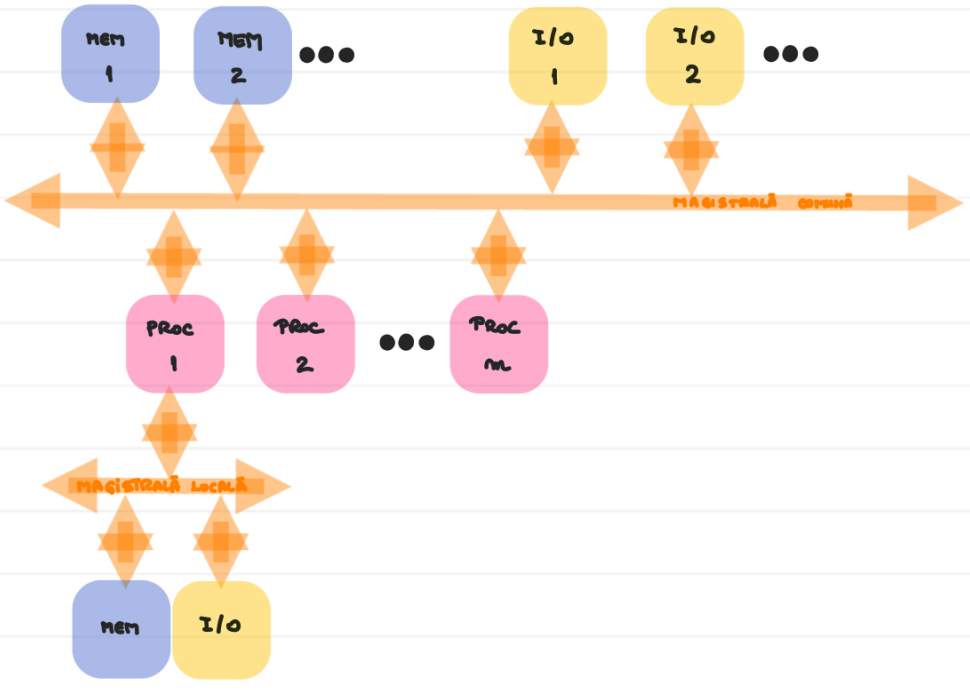



b) locală - dacă se strică o linie, doar ea nu va mai funcționa

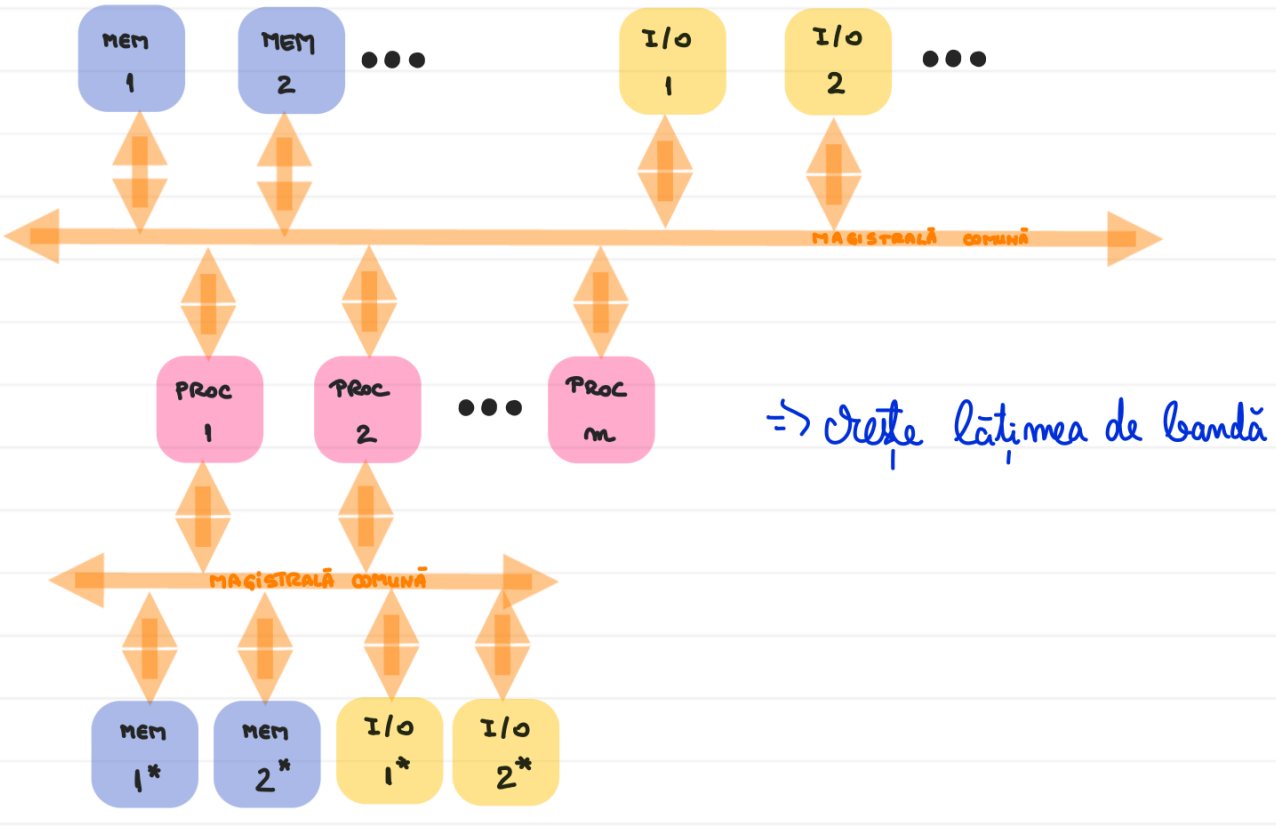


13. Dați exemple pentru un sistem multiprocesor cu resurse partajate.

A) Sistem multiprocesor cu resurse comune (partajate) și resurse locale




B) Sistem multiprocesor cu 2 seturi de resurse comune (partajate)




=> crește lățimea de bandă

20. Care este principalul dezavantaj al unui sistem multiprocesor cu resurse partajate și ce soluție există pt. gestionarea acestuia?

Principalul dezavantaj al unui sistem multiprocesor este accesul simultan la magistrala comună, ceea ce ar duce la erori de funcționare.

Va trebui stabilită o ordine de prioritate, a.î. cel mult un procesor să aibă acces să-și continue activitatea, iar celelalte să fie trecute în stări de așteptare

⇒ apare problema arbitrării cererilor de acces

O soluție ar fi utilizarea circuitului arbitru (pt 8086 este 8289). Acesta activează linia $\overline{AEN}$ din pl pt. a permite accesul pe magistrală.

Există 3 soluții de prioritizare : - paralelă
- serială
- rotativă

21. Este necesară existența unui decodificator de memorii într-un microsistem digital? Justificați răspunsul.

Da este necesară existența unui decodificator de memorie într-un microsistem digital, deoarece acesta are rolul de a genera semnale de selecție (CS - Chip Select) pt. zonele de memorie specifice.

Procesorul va pune pe magistrala de adrese adresa unde va dori să lucreze, iar decodificatorul va activa memoria în care este localizată adresa.

În plus, decodificatorul de memorie reduce conflictele de pe mag. de date, prin semnalele de validare și selecție.

# 22. Descrieți decodificarea completă. Avantaje și dezavantaje

Se construiește un tabel – pe coloane : rangurile de adrese
– pe linii : configurații care arată adresa de început și de sfârșit a fiecărei zone ocupate de un circuit

Pentru fiecare circuit se inspectează coloanele, pornind de la rangurile mai semnificative spre cele mai puțin semnificative.

Se stabilesc acele ranguri ce rămân nemodificate oricare ar fi locația adresată în zona respectivă.

Se scrie o funcție combinațională a acestor intrări ce va individualiza zona de memorie respectivă și va constitui ecuația pt. semnalul de selecție al circuitului pe care îl acoperă.

Avantaje → acces unic : fiecare circuit (locație de memorie) are o adresă unică
→ izolare completă : se asigură integritatea datelor
→ generalitate : poate fi aplicată oricărei hărți de memorie

Dezavantaje → complexitate crescută : nr. mare de linii de adresă
→ dificultăți la extindere : la adăugarea de componente noi, trb. modificat decodificatorul

# 23. Descrieți decodificarea incompletă. Avantaje și dezavantaje

Se construiește un tabel - pe coloane : rangurile de adrese
- pe linii : configurații care arată adresa de început și de sfârșit a fiecărei zone ocupate de un circuit

Pentru fiecare circuit se inspectează coloanele, pornind de la rangurile mai semnificative spre cele mai puțin semnificative.

Se stabilesc acele ranguri ce rămân nemodificate oricare ar fi locația adresată în zona respectivă.

O funcție combinațională care va genera semnalele de selecție nu va utiliza toate liniile de adresă ce rămân nemodificate oricare ar fi locația adresată din zona respectivă

Dacă nu se consideră o linie -> va ocupa 2 zone ; 2 linii -> 4 zone

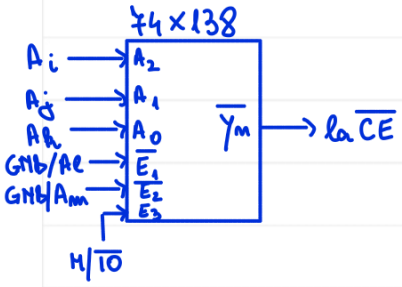
Avantaje: → reducerea complexității : mai puține linii de adresă
→ economie: mai puține circuite
→ adecvată pt aplicații cu cerințe reduse

Dezavantaje: → conflict de adresare : pot apărea situații în care mai multe locații accesează aceeași componentă
→ extinderea memoriei poate fi imposibilă dacă fiecare circuit și-a multiplicat prezența => trb să se refacă decodificatorul
→ izolare dificilă a componentelor

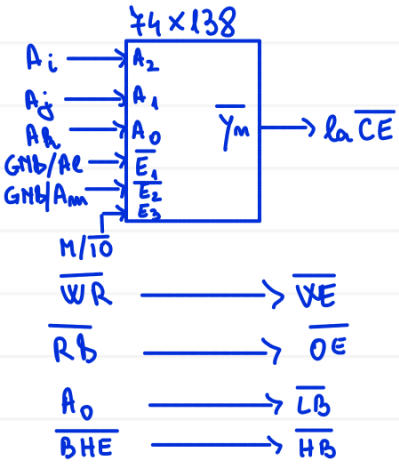
24. Care este diferența între soluțiile pentru conectarea unei memorii fixe, SRAM sau BRAM într-un microsistem digital?

Conectarea memoriilor într-un microsistem digital diferă prin scopul și cerințele lor.

A) Memoria fixă → mem. de tip ROM
→ păstrează datele permanent
→ nu necesită reîmprospătare
} avem o conectare mai simplă, cu un singur semnal de comandă
→ necesită un decodificator simplu, datele sunt doar citite
→ compatibilă CMOS, TTL




B) SRAM → memorie rapidă
→ nu necesită reîmprospătare
→ datele se păstrează cât memoria este alimentată
→ conectată la o magistrală de adrese și una de date, cu circuite suplimentare scrierii și citirii
→ viteză mare 12/15 ns
→ i/O compatibile TTL, cu 3 stări




C) BRAM → memorie dinamică
→ necesită reîmprospătare la 2-3 ms pt. a păstra datele => ne trebuie un controller BRAM; dacă se fac accese simultane din partea μP și controlerului, acesta va avea prioritate
→ folosește mai puțini tranzistori per celulă de memorie; capacitatea mare înseamnă grad de integrare mare și cost pe bit memorat mic

25. Se pot conecta porturi la un procesor care nu are sistem de intrare/ieșire? Dacă nu, de ce? Dacă da, cum?

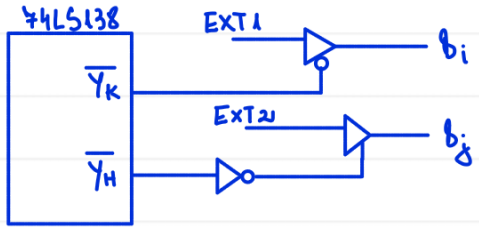
Da, se pot conecta porturi la un procesor care nu are sistem de intrare/ieșire, dar acestea trebuie legate în spațiul de memorie.

μP le va trata ca blocuri de memorie pentru că nu va face distincția între porturi și memorie. Comunicarea se va realiza deci prin instrucțiuni de lucru cu memoria. În sistem va exista decodificatorul de memorie, iar porturilor le vor fi alocate zone din spațiul de memorie/adrese.

Magistrala de adrese va fi utilizată pt selecție, iar cea de date pt. transmiterea informației.

Se reduce nr. de terminale, dar zona de memorie va fi ocupată de porturi.

2.6. Prezentați și comentați soluția pentru portul realizat cu porți. Pentru ce tip de porți se recomandă? De ce?



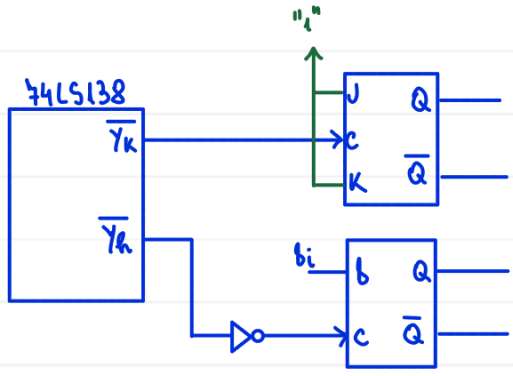

Este folosit pentru porturile de intrare pentru că permite citirea eficientă a semnalelor externe prin magistrala de date și pentru că se respectă regula că la magistrală nu se leagă porți fără facilitatea de a 3-a stare.

Semnalele care vin din exterior se p.p. că sunt memorate de circuitul care le generează până când este necesar. Ele sunt trecute printr-o poartă cu 3 stări și apoi ajung pe o linie din magistrala de date, de unde vor fi preluate și procesate.

O ieșire a decodificatorului activează sau nu poarta.

Dacă se execută instrucțiunea aferentă ieșirii folosite, cu adresa de port aferentă ieșirii folosite atunci se activează ieșirea și se deschide poarta, informația trecând mai departe.

27. Prezentați și comentați soluția pentru portul realizat cu bistabile. Pentru ce tip de port se recomandă? De ce?




| J | K | $Q_{m+1}$ |
|---|---|---|
| 0 | 0 | $Q_m$ |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | $\overline{Q_m}$ basculare |

Este folosit pentru porturile de ieșire, deoarece semnalele de la exterior trebuie să persiste cevașa timp pt. a fi captate (ex: aprinderea unui LED). De aceea, portul cu poartă nu este fiabil. Bistabilele au capacitatea de a memora informația și de a o menține activă spre exterior până la următoarea actualizare.

a) Bistabil JK

Se va schimba valoarea curentă în registru, nu se poate fixa o valoare, iar la fiecare comandă de ieșire, se generează un impuls și bistabilul își schimbă starea. Sunt mai dificil de utilizat, deoarece comutarea stării necesită cunoașterea stării anterioare.

b) Bistabil D

Permite să conectăm linia D la o informație, care va apărea și la ieșire, în momentul generării unui impuls de tact pentru registru. Sunt preferate deoarece ieșirea preia direct starea bitului D transmis de procesor, fără a fi nevoie să se știe starea anterioară.

| D | $Q_{m+1}$ |
|---|---|
| 0 | $Q_m$ |
| 1 | $\overline{Q}_{m+1}$ |

# 28. Ce înseamnă comanda unui semnal prin program?

1. Citirea unui semnal din ext. microsistemului
   → soluția este citirea semnalului prin intermediul unui port implementat cu porți
   → semnalul este legat la intrarea unei porți cu 3 stări
   → poarta este activată sau nu printr-un semnal provenit de la un decodificator de porturi
   → ieșirea porții este legată la o linie a magistralei de date a UC
   → citirea se face cu o instrucțiune IN, apoi se face ramificarea

2. Poziționarea unui semnal la "1" sau "0" logic
   → soluția este implementarea unui port cu bistabile
   → valoarea semnalului este transmisă la intrarea de date a bistabilului printr-o linie a magistralei de date
   → tactul bistabilului este comandat de o ieșire a decodificatorului de porturi
   → ieșirea bistabilului e un semnal care trebuie comandat.
   → poziționarea semnalului se face cu o instrucțiune OUT.

29. Ce înseamnă circuit specializat programabil? Programabilitatea sa coincide cu cea a unui procesor? Dacă nu, care este diferența?

Un circuit specializat se referă la standardizarea soluțiilor pentru diversele tipuri de aplicații rezolvabile cu sisteme cu microprocesoare.

Cerințele sunt → flexibilitatea - pentru a acoperi toate modurile de lucru
→ programabilitatea

Un circuit specializat este programabil dacă este capabil, prin intermediul unuia sau a mai multor cuvinte de comandă, să aleagă particularități ale tipului de aplicație pentru care circuitul a fost conceput. Cuvintele de comandă sunt transmise de UC.

Programabilitatea sa nu coincide cu cea a unui procesor, deoarece acesta (μP) știe să execute cod. CSP știe doar să înțeleagă și să se conformeze față de ce cuvânt de comandă i s-a transmis. CSP nu rulează un program, ci funcționează conform configurației stabilite.

Unele CSP oferă μP informații de stare legate de ultimul transfer, dacă au fost probleme și eventual le diagnostichează.

# 30. Prezentați și comentați caracteristicile de bază ale interfeței RS232.

Interfața serială RS232 este cea mai veche dar și cea mai răspândită.
Nivelele de tensiune sunt EIA nu TTL adică: -25V -> 3V pt "1" logic
3V -> 25V pt "0" logic

Sunt necesare circuite de conversie TTL -> EIA și EIA -> TTL
Alt avantaj îl reprezintă direcția de deplasare a datelor care poate fi:

1. simplex (unidirecțională)
2. half-duplex (bidirecțională dar la momente de timp diferite)
3. full-duplex (bidirecțională ce poate avea loc simultan)

Viteza este mică, dispusă în trepte, atinge până la 0,5 Mps. Există 2 unități de măsură a vitezei: bps și baud. Dacă transferul este digital 1 bps = 1 baud. Câteva trepte de viteză mai utilizate: 300, 600, 1200, 2400, 4800, 9600, 19200, 38400, 57 600, 115 200, mai rar 230 400 sau 460 800.

Este util când există distanțe mari (>3m) între echipamentele care comunică.
→ cost redus dacă nr de cabluri e redus
→ rezistență mai mare la perturbații față de transferul paralel

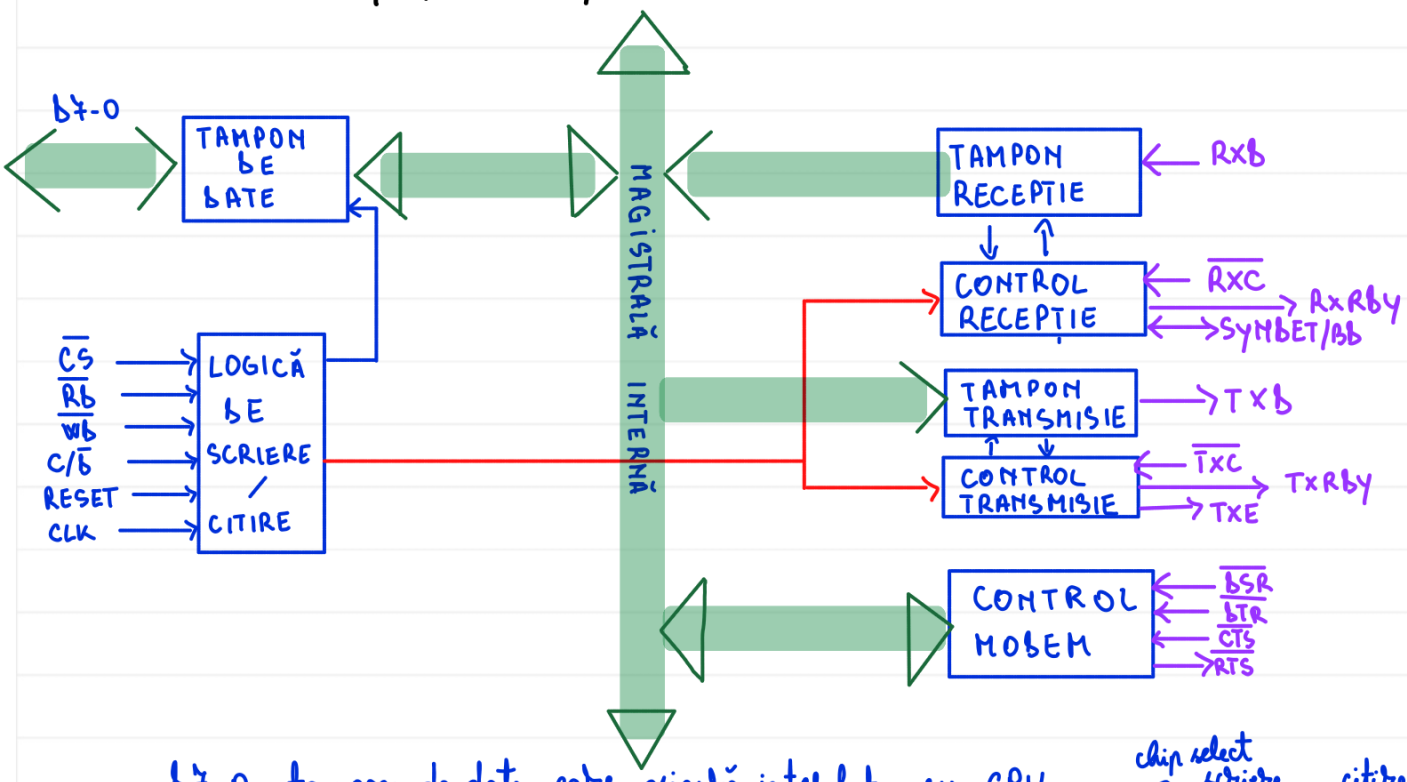
31. Ce este circuitul 8251? Prezentați caracteristicile sale.

Circuitul 8251 este un circuit specializat programabil pentru transferurile seriale de tip RS232. Face parte din categoria circuitelor de tip USART ("Universal Synchronous Asynchronous Receiver Transmitter) și are 2 moduri de lucru: sincron și asincron.

Are capacitatea de a prelua un octet în paralel de la UC, să-l serializeze și să-l transmită la un echipament serial. Poate să preia de pe linie de la un echipament serial, un octet, să-l asamblează și să-l predea în paralel UC.

8251 comunică UC când are un caracter gata pt. ea sau când a terminat de transmis un octet și poate prelua altul. Poate comunica prin program sau prin întreruperi. Are transmisie și recepție cu dublu tampon.

# 32. Prezentați și comentați structura internă a circuitului 8251.




b7-0 tampon de date care asigură interfața cu CPU

Logică de scriere/citire → gestionează semnalele de control ($\overline{CS}$, $\overline{WR}$, $\overline{RD}$) (chip select, scriere, citire)

→ C/$\overline{D}$ (control/data) selectează portul de comandă/date

## Bloc de recepție (tampon + control)

→ monitorizează linia serială, asamblează octetul pt CPU

→ RxD - linia serială pt date

→ RxRDY - anunță procesorul că are un caracter valid

→ $\overline{RXC}$ - tact de recepție

## Bloc de transmisie (tampon + control)

→ preia octetul din tampon și îl trm serial

→ TxD - linia pe care pleacă date

→ TxRDY - anunță procesorul că tamponul e gol și poate prelua alt caracter

## Control modem

→ gestionează semnalele de handshaking

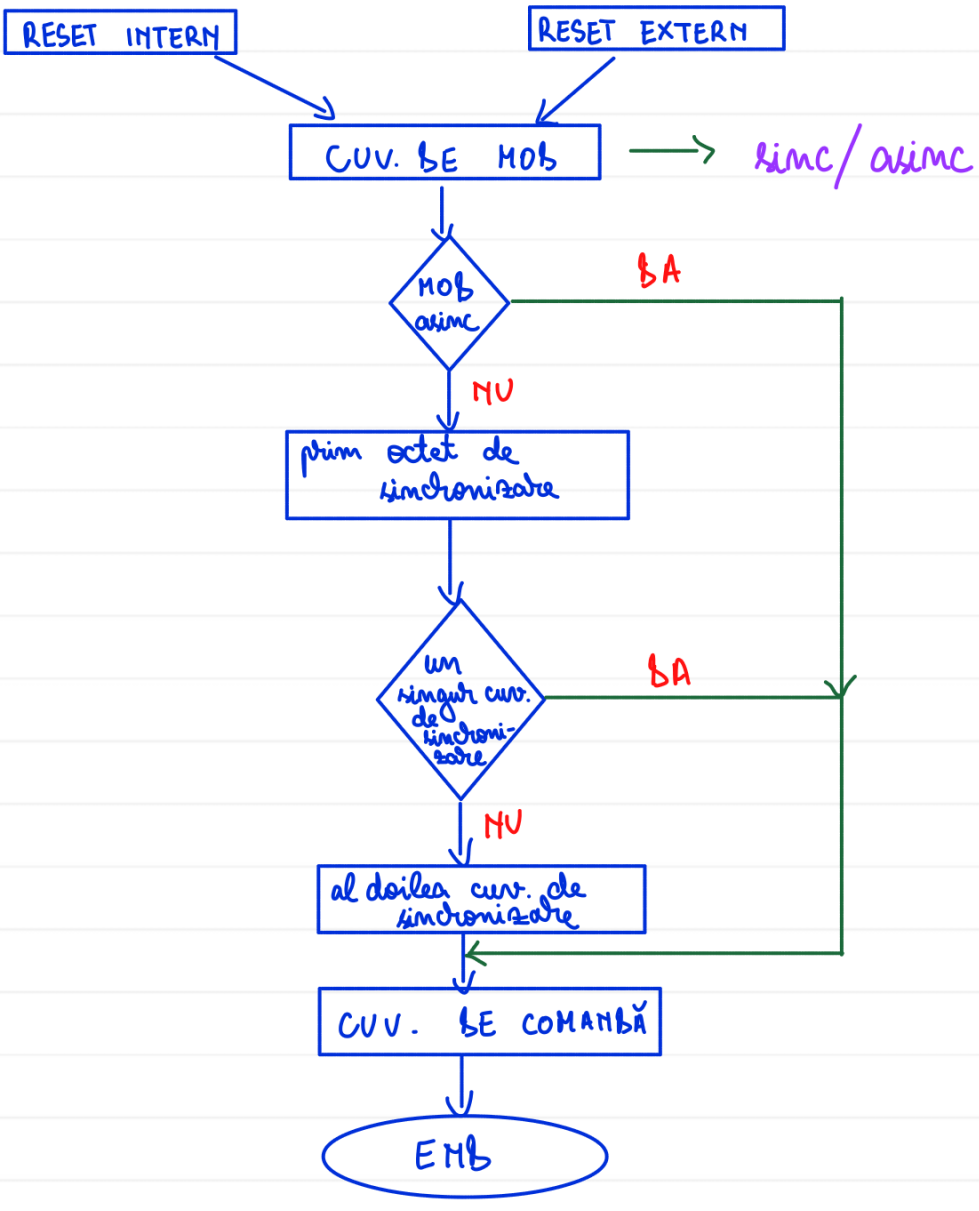
$\overline{DTR}$ - data terminal ready → μP

$\overline{DSR}$ - data set ready → modem

$\overline{RTS}$ - req to send → μP

$\overline{CTS}$ - clear to send → perif.

33. Cum se programează circuitul 8251 ( nu este necesara structura cuvintelor ci doar ce anume i se comunică circuitului )?




Circuitul 8251 este specializat pentru transferurile seriale. Este de tip USART, are 2 moduri sincron si asincron. Programarea circuitului(dupa un reset) se realizeaza prin scrierea unor cuvinte de control. Prima data este cuvantul de mod. Pe langa mod se alege cadrarea, bitii de paritate, factorul de multiplicare si lungimea de carcater. Daca modul ales este asincron se trece direct la cuvantul de comanda.  Daca modul este sincron se aleg unul sau 2 covinte de sincronizare. De asemenea prin cuvantul de stare se exprima starea circuitului intr-un anume moment.

C7

## 34. Ce este circuitul 8253? Prezentați caracteristici ale sale.

Circuitul 8253 este un CSP de numărare și temporizare, utilizat pentru numărarea evenimentelor și contorizarea timpului.

Caracteristici

→ dispune de 3 contoare identice, independente pe 16 biți, cu preîncarcare, numără în jos
  → fiecare contor poate lucra în 6 moduri

→ structura sa include:
  → tamponul de date
  → logica de scriere / citire
  → RCC - registru de comandă și scriere
  → o magistrală internă
  → cele 3 contoare

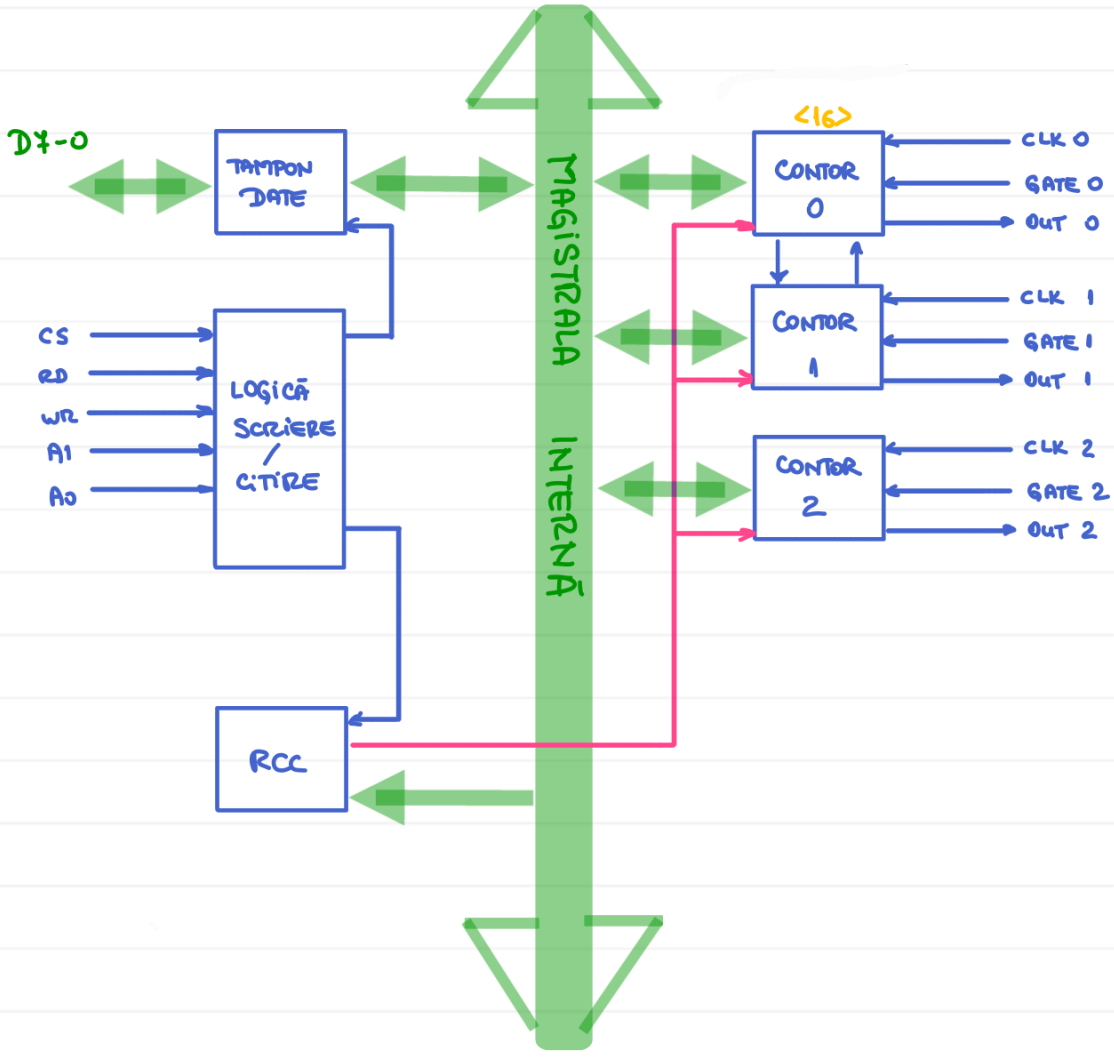
→ conținutul contoarelor poate fi citit de pP

Dacă se nr. evenimente, impulsurile care le descriu se leagă la CLK.
Dacă se dorește generarea de întârzieri se leagă tactul sistemului

Modurile → 0 → generarea unei cereri de întreruperi
→ 1 → monostabil programabil
→ 3 → generator de semnal dreptunghiular

C7

# 35. Prezentați și comentați structura internă a circuitului 8253.




Tamponul de date asigură interfața dintre circuit și magistrala de date

Logica de scriere/citire

→ conduce intreg transferul de date din int. circuitului

→ CS, RD, WR îi spun ce fel de operație se face

→ A1 și A0 selectează după urm tabel

| A1 | A0 | SEL |
|---|---|---|
| 0 | 0 | C0 |
| 0 | 1 | $C_1$ |
| 1 | 0 | $C_2$ |
| 1 | 1 | RCC |

Registrul de comandă și control

→ preia cuv. de comandă transmise de μP care arată contoarele implicate în op. curente și modurile de lucru ale acestora

→ este WO (write only)

# Contoarele

→ sunt numărătoare pe 16 rangură, preîncărcabile

→ numără în jos, având ca intrare linia $CLK_i$

→ CLK = tact sistem ⇒ nr. timpul

→ CLK = venit de la un traductor care sesizează că a apărut un eveniment ⇒ nr. evenimente

→ sunt identice și independente

→ conținutul poate fi citit

36. Cum se programează circuitul 8253 (nu este necesară structura cuvintelor ci doar ce anume i se comunică circuitului)?

Fiecare dintre cele 3 contoare ale circuitului va fi programat individual, pp prividu-l ca un port de i/O.

Fiecare contor va fi programat cu un cuvânt de comandă și cu o constantă de preîncărcare de 1 sau 2 octeți.

În cuv. de comandă se primește:

→ cui se adresează cuv → contor nr. (biți 7-6)

→ ce vrem de la el → scriere/citire (biți 5-4)

→ modul de lucru (biți 3-1)

→ 0 → generarea unei cereri de întreruperi

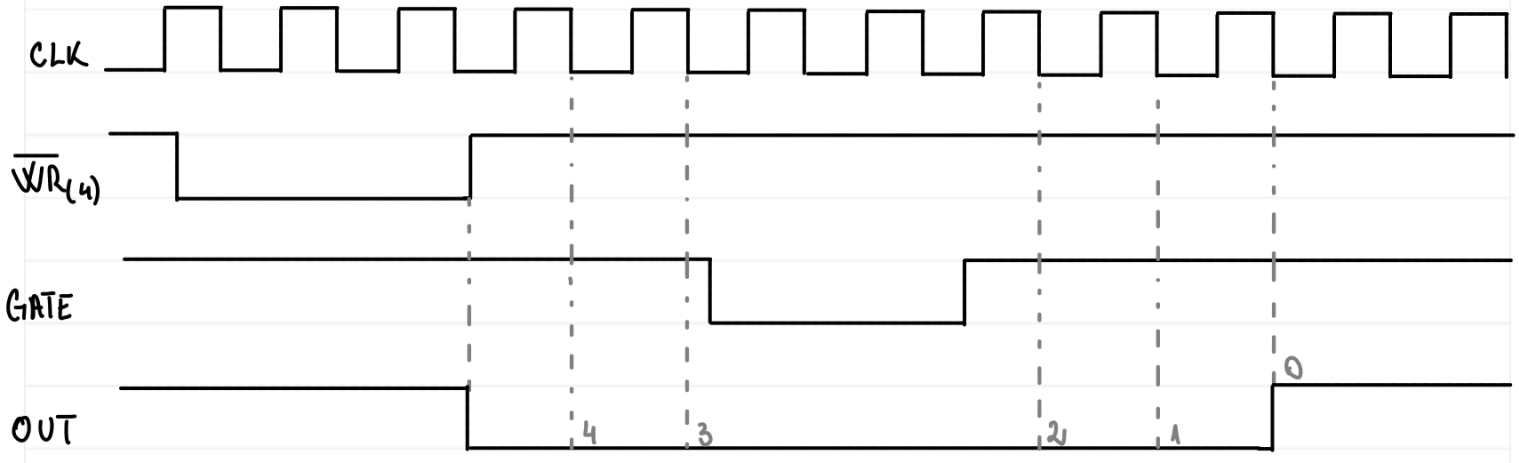
→ 1 → monostabil programabil

→ 3 → generator de semnal dreptunghiular

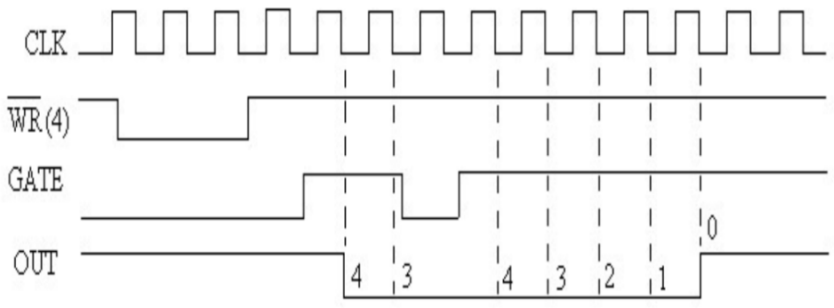
→ cum se primește informația → binar/BCD (bit 0)

# 37. Descrieți un mod de lucru al unui contor al circuitului 8253.

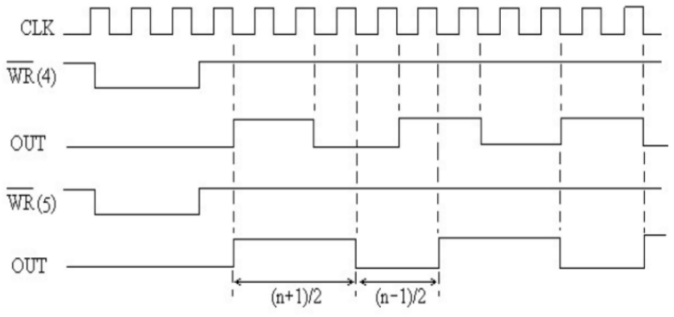
Modul 0 sau generarea unei cereri de întrerupere la sfârșitul numărării
→ după programarea contorului, ieșirea trece pe nivel „0" logic
→ se menține la această valoare până când contorul va atinge valoarea 0, în urma numărării impulsurilor venite pe linia CLK, când va trece la 1 logic
→ rămâne în această stare până la o nouă încărcare
→ numărarea poate fi inhibată prin activarea intrării GATE.
→ decrementarea începe după încărcarea constantei și după trecerea unui impuls




- **Modul 1** sau monostabil programabil:
  - Ieşirea trece în starea 0 logic:
    - după ce are loc încărcarea contorului,
    - după ce are loc o tranziţie pozitivă pe linia GATE şi
    - după ce trece un impuls pe linia CLK.
  - Ieşirea rămâne la 0 logic până când contorul ajunge la valoarea 0 după care aceasta revine la 1 logic;
  - Dacă, în timpul numărării, este încărcată în contor o nouă valoare aceasta nu va fi luată în considerare dacă nu există tranziţie pozitivă pe linia GATE; o astfel de tranziţie va relua procesul de decrementare de la valoarea iniţială chiar dacă numărarea nu s – a încheiat, iar dacă înaintea tranziţiei contorul a fost înscris cu o nouă constantă, decrementarea va avea loc în conformitate cu noua constantă.
  - Modul 1 se mai numeşte şi monostabil programabil întrucât numărarea este anclanşată de o tranziţie pe un semnal întocmai ca la un monostabil la care anclanşarea este comandată tot de o tranziţie pe un semnal.
  - Fig. prezintă variaţia ieşirii unui contor în modul 1.



- **Modul 3** sau generator de semnale dreptunghiulare:
  - Ieşirea contorului va genera semnal dreptunghiular cu perioada egală cu perioada tactului înmulţită cu valoarea constantei;
  - Factorul de umplere al semnalului va fi 1/ 2 dacă constanta este pară iar dacă aceasta este impară, semnalul va avea 1 logic o durată egală cu (n + 1)T/ 2 şi 0 logic o durată (n – 1)T/ 2, unde n este valoarea constantei iar T este perioada tactului;
  - Fig. prezintă variaţia ieşirii unui contor în modul 3:



38. Câte linii sunt necesare pentru un transfer paralel? Justificați.

În funcție de dimensiunea entității care se transferă:

→ 8b : 8 + 2 + 1 = 11 linii

→ 16b : 16 + 2 + 1 = 19 linii

⇒ n+3 linii

Considerăm o entitate de n biți care se dorește a fi transferată paralel. Vom utiliza o interfață paralelă cu

→ n linii de date

→ 2 linii de dialog ( 1 de la transmițător la receptor și alta invers)

→ cel puțin o linie de masă

⇒ avem nevoie de cel puțin n+3 fire pt. o comunicare paralelă

# 39. Ce este circuitul 8255? Prezentați caracteristici ale sale.

Circuitul 8255 este un CSP pentru transferurile paralele.

Dispune de 24 de linii de i/o care pot fi configurate în mai multe feluri, în funcție de modul de lucru ales:

→ 2 grupe de câte 12 linii de i/o, fără semnale de dialog
→ 2 grupe de câte 8 linii de i/o, cu semnale de dialog $\left(A+\frac{C}{2} / B+\frac{C}{2}\right)$
→ o grupă de 8 linii bidirecționale, cu semnale de dialog

Din cele 24 de linii de i/o doar 16 au posibilități de memorare

Comunicarea cu circuitul 8255 se face prin intermediul a 4 adrese de port, corespunzătoare porturilor A, B, C și portului pt. cuv. de comandă.

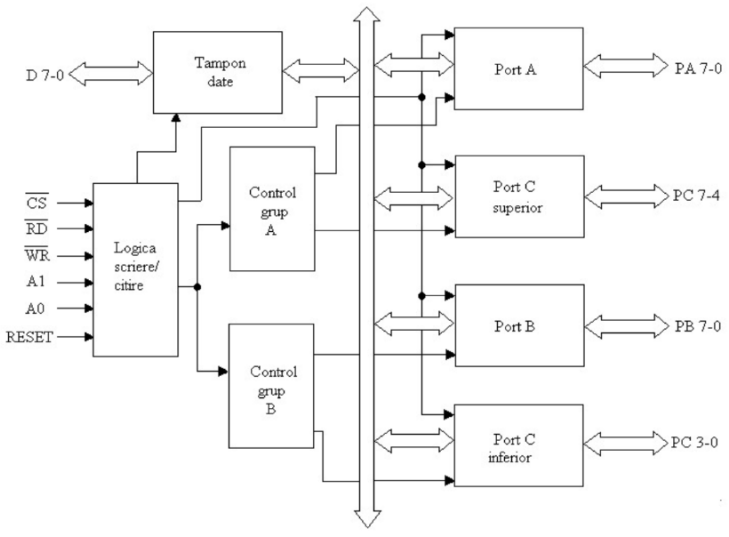
Are 3 moduri

→ mod 0 :→ i/o simplă, fără semnale de sincronizare
→ datele sunt citite sau scrise pt A, B și C
→ mod 1: → i/o cu dialog pt A și B
→ mod 2: → mod bidirecțional cu dialog doar pt A

Compatibilitate cu TTL

# 40. Prezentați și comentați structura internă a circuitului 8255.

- Structura internă:




  - A1 și A0 sunt intrări pentru selectarea unuia din cele 4 porturi interne ale circuitului: 00 pentru Port A, 01 pentru Port B, 10 pentru Port C și 11 pentru RCC; se leagă linii din mag. de adrese (de obicei A2 și A1);

Tampon de date: asigură legătura dintre circuit și magistrala de date a sistemului: transferă cuvinte de comandă, de stare și date

Logica de scriere/citire are rolul de a conduce întregul transfer de date din interiorul circuitului.

→ are ca intrări semnale de comandă de la μP care îi comunică ce fel de op. se desfășoară

→ comandă cele 2 jumătăți ale circuitului grupul A și grupul B

→ blocurile de control pt. grupele A și B primesc comenzi de la blocul Logică scriere/citire, primesc cuv. de comandă și generează comenzile pt porturile corespunzătoare

Porturile A, B, C au câte 8 biți:

1) portul A → ieșire în modurile 0, 1, 2
   → cu posibilități de memorare și intrare în modurile 0, 1, 2
   → cu posibilități de memorare doar în modul 1 sau 2
2) portul B → ieșire în modurile 0, 1
   → cu posibilități de memorare și intrare în modurile 0, 1
   → cu posibilități de memorare doar în modul 1
3) portul C → ieșire în modurile 0
   → cu posibilități de memorare și intrare în modurile 0
   → fără posibilități de memorare

→ dacă A, B → mod 1 sau 2 ⇒ rangurile din portul C vor fi folosite ca semnale de dialog

⇒ poate fi folosit pt. a obține informații de stare de la circuit

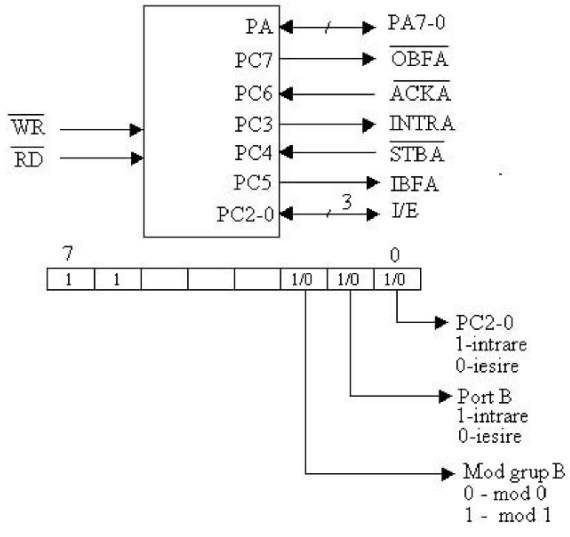
C8/p6

41. Cum se programează circuitul 8255 (nu este necesară structura cuvintelor ci doar ce anume i se comunică circuitului)?

- Se realizează prin transmiterea unui cuvânt de comandă la adresa portului cuvântului de comandă; se poate cere porturilor să lucreze în unul din următoarele moduri:
  - modul 0, numit şi mod intrare/ ieşire, pentru porturile A, B şi C,
  - modul 1, numit şi mod intrare/ ieşire cu dialog, pentru porturile A şi B şi
  - modul 2, numit şi mod bidirecţional cu dialog, doar pentru portul A.
- Există şi posibilitatea ca, printr – o singură instrucţiune de ieşire, microprocesorul să comande ieşiri individuale ale portului C; se face printr – un cuvânt de comandă în care rangul 7 este la valoarea 0 logic; particularitatea este cunoscută sub numele mod “bit set/ reset” şi este utilă în aplicaţii de control.
- O altă facilitate: generarea unui semnal care poate fi folosit ca cerere de întrerupere; va fi generat în anumite condiţii, care vor fi prezentate, şi, în plus, trebuie validat sistemul de întreruperi al circuitului; constă în câte un bistabil, pentru modurile de lucru 1 şi 2, iar activarea sau inactivarea bistabilului se face prin ranguri din portul C.

# 42. Descrieți unul din modurile de lucru cu dialog ale circuitului 8255.

- Modul 2:
- Se mai numeşte şi mod bidirecţional cu dialog; semnalele de dialog au aceleaşi semnificaţii ca în modul 1, poziţiile lor relative sunt aceleaşi, existând, totuşi, o diferenţă: semnalul /ACK, la 0 logic validează transmiterea datelor de către portul A către periferic iar dacă este 1 logic liniile de date vor fi în a 3 – a stare; /ACK nu mai are semnificaţie de achitare, ci semnificaţie de cerere. Caracteristici:
  - doar grupul A poate lucra în acest mod,
  - grupul A este alcătuit din portul A şi 5 semnale de dialog care folosesc ranguri ale portului C,
  - grupul B poate fi programat în modul 1 sau 0,
  - există facilitatea de memorare şi la intrare şi la ieşire,
  - portul C are şi rol de registru de stare,
  - Sistemul de întreruperi este implementat de bistabilele INTE1 şi INTE2. INTE1 este asociat ieşirii şi este comandat prin terminalul PC6 iar INTE2 este asociat intrării şi este comandat de terminalul PC4.
- Terminalele portului C folosite de grupul A pentru dialog:



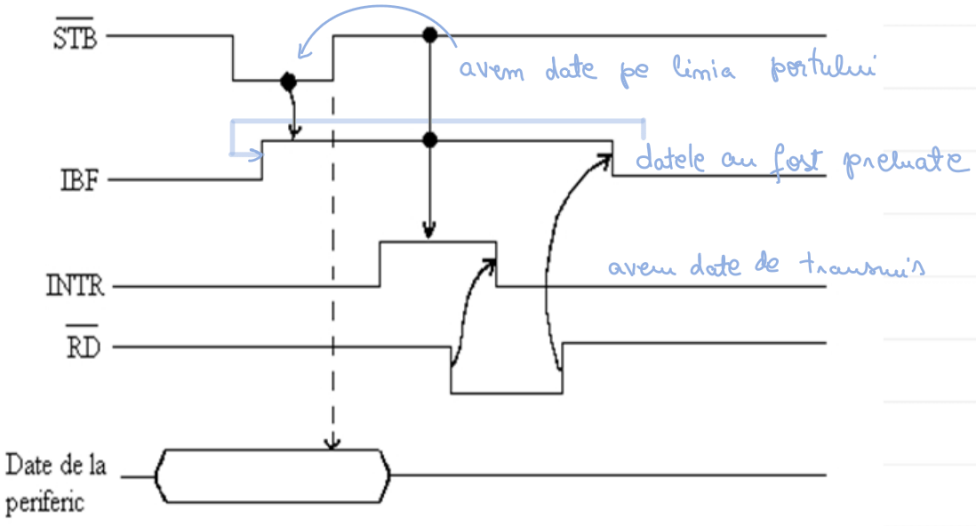

- Cuvântul de stare în modul 2:

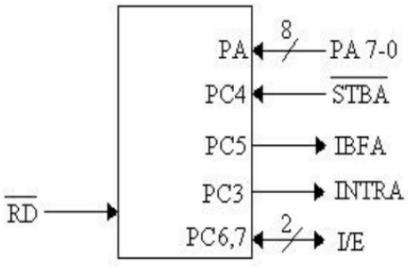
| 7 | | | | | | | 0 |
|---|---|---|---|---|---|---|---|
| /OBFA | INTE1 | IBFA | INTE2 | INTRA | I/E | I/E | I/E |

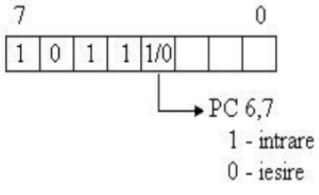
- Modul 1:

- Se mai numeşte şi mod de intrare/ ieşire cu dialog şi se foloseşte pentru operaţii de intrare/ ieşire cu dialog. Caracteristicile modului sunt următoarele:
  - se pot comanda independent grupul A şi grupul B;
  - grupul A conţine portul A şi 3 ranguri ale portului C iar grupul B conţine portul B şi alte 3 ranguri ale portului C;
  - rangurile nefolosite ale portului C pot fi comandate prin modul bit set/ reset;
  - porturile A şi B pot fi declarate ca intrări sau ieşiri având şi posibilităţi de memorare;
  - portul C are şi rol de registru de stare.

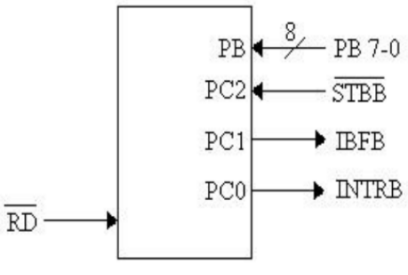
- Semnalele de dialog folosite în modul 1 intrare sunt:
  - /STB (“Strobe”): intrare prin care perifericul anunţă că a plasat date pe liniile portului; 0 logic va provoca încărcarea datelor în port;
  - IBF (“Input Buffer Full”): ieşire, activă la 1 logic, prin care portul anunţă perifericul că datele au fost preluate; semnalul redevine inactiv după ce semnalul /RD de la microprocesor devine inactiv, ceea ce corespunde situaţiei în care microprocesorul a preluat data încărcată cu /STB;
  - INTR (“Interrupt”): este o ieşire, activă la 1 logic, către microprocesor, prin care 8255 îi comunică acestuia că un port de intrare are o dată de transmis; semnalul poate fi cerere de întrerupere pentru microprocesor; semnalul va fi activ când /STB = 1, adică data a fost încărcată, IBF = 1, adică i s – a comunicat perifericului că data a fost preluată şi dacă sistemul de întreruperi al circuitului este activ; semnalul va fi dezactivat de frontul anterior al semnalului /RD care corespunde situaţiei în care microprocesorul a luat în considerare cererea; sistemul de întreruperi este implementat cu bistabilul INTEA pentru portul A şi INTEB pentru portul B şi este comandat în modul bit set/ reset prin rangurile PC4 pentru portul A, respectiv PC2 pentru portul B ale portului C.

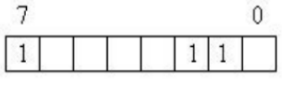


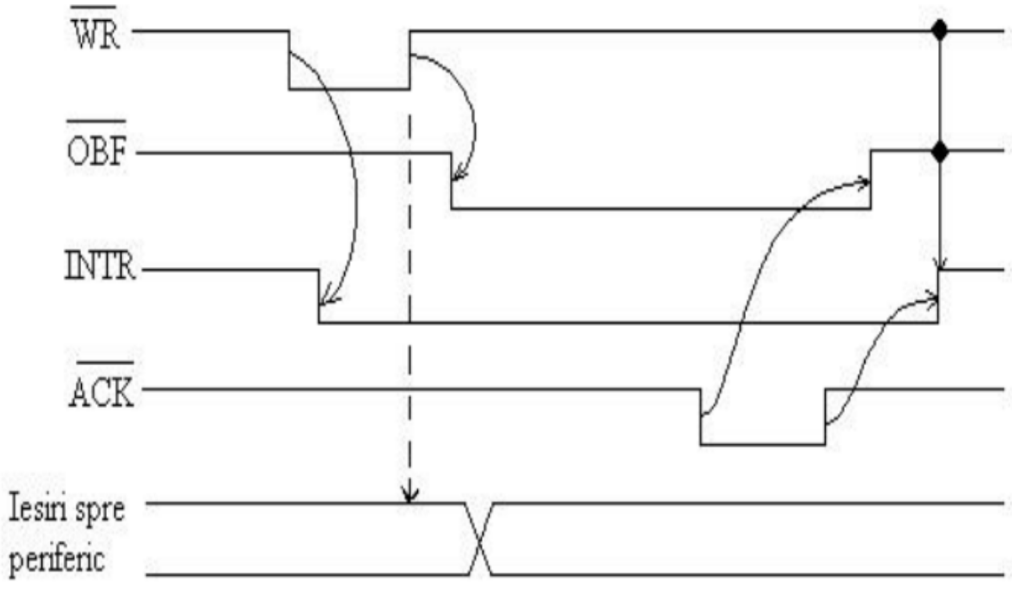

Grup A in mod 1 intrare




Grup B in mod 1 intrare

- Semnalele de dialog folosite în modul 1 ieşire sunt:
  - /OBF (“Output Buffer Full”): ieşire, activă la 0 logic, prin care portul anunţă perifericul că microprocesorul i – a trimis o dată şi a plasat – o pe liniile proprii; semnalul este activat după dezactivarea lui /WR, prin care microprocesorul înscrie data în port, şi va fi dezactivat de răspunsul perifericului;
  - /ACK (“Acknowledge”): intrare, activă la 0 logic, prin care perifericul anunţă că a preluat data trimisă de portul A sau B; este răspunsul la semnalul /OBF;
  - INTR (“Interrupt”): ieşire, activă la 1 logic, către microprocesor, prin care 8255 îi comunică acestuia că a transmis către periferic data primită de la el şi aşteaptă o nouă dată; semnalul poate fi cerere de întrerupere pentru microprocesor; semnalul va fi activ când /OBF = 1, adică data a fost transmisă către periferic, /ACK = 1, adică perifericul a preluat data şi dacă sistemul de întreruperi al circuitului este activ; sistemul de întreruperi este implementat cu bistabilul INTEA pentru portul A şi INTEB pentru portul B şi este comandat în modul “bit set/ reset” prin rangurile PC6 pentru portul A, respectiv PC2 pentru portul B ale portului C.



43. Care dintre cele 3 posibilități de conectare a perifericelor la o UC cu microprocesor este mai rapidă?

1. Intrare/ieșire programabilă (PIO) - se mai numește pooling
   → se desfășoară prin intermediul UC
   → fiecare periferic / port are un indicator (care poate fi realizat cu un bistabil) prin care anunță dacă vrea să transfere ceva cu procesorul
   → e o soluție simplă, nu cere resurse scumpe
   → se citesc în buclă indicatorii; se găsește unul activ → transfer
   → timpul procesorului este ineficient folosit

2. Întreruperi
   → procesorul este întrerupt la transferul fiecărui cuvânt
   ⇒ apar multe întreruperi care nu pot fi mereu acceptate
   ⇒ rata de transfer devine limitată

3. Accesul direct la memorie (DMA)
   → transfer direct | mem - periferic
   | mem - mem
   | periferic - periferic
   → nu implică UC, dar cere hardware suplimentar un controler DMA conectat prin magistrale
   → transferul DMA se desfășoară în cadrul unui canal DMA accesibil prin adrese de port

⇒ DMA este cel mai rapid

## 44. Ce înseamnă DMA și ce resurse cere?

DMA este o metodă de transfer de date care permite perifericelor accesul direct la memoria principală, fără intervenția µP, pentru a crește viteza de transfer de date și a reduce încărcarea µP

Caracteristici:

→ transfer direct | mem - periferic
| mem - mem
| periferic - periferic

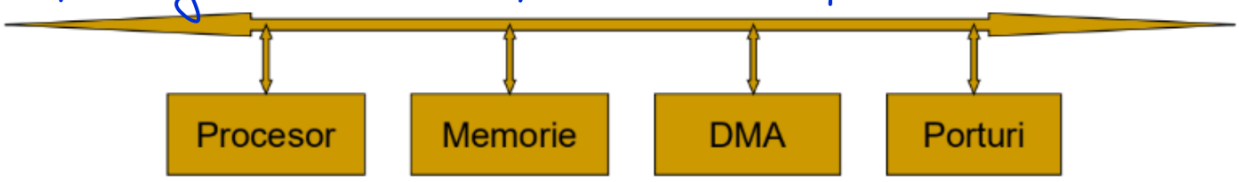
→ nu implică UC, dar cere hardware suplimentar un controler DMA conectat prin magistrale

→ transferul DMA se desfășoară în cadrul unui canal DMA accesibil prin adrese de port

Un controler DMA → preia controlul magistralei de la µP
→ perifericul transmite / primește date direct prin mem. princ.
→ µP este modificat de finalizarea transferului printr-o întrerupere.

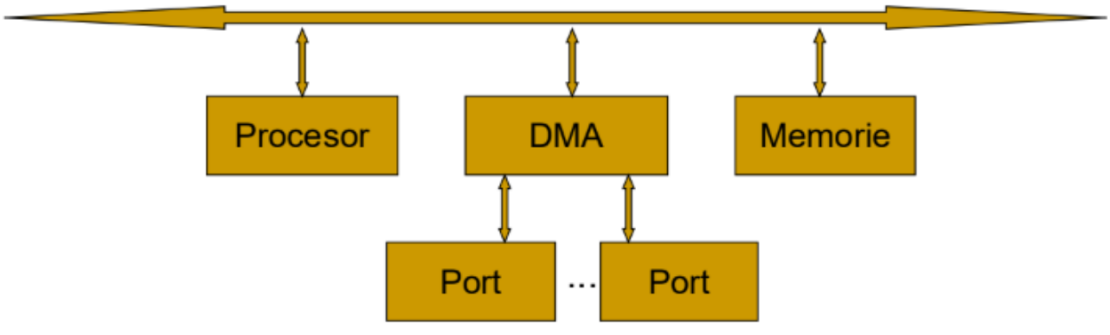
45. Scrieți 3 configurații DMA.

A) Magistrală comună, controler DMA separat



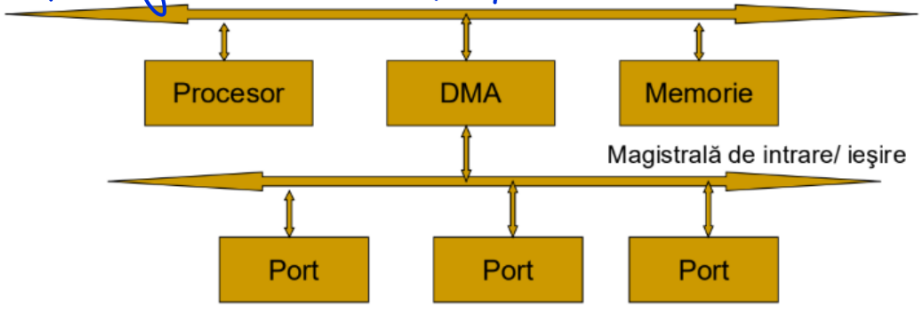

→ fiecare transfer folosește magistrala de 2 ori
→ port cu DMA
→ DMA cu memoria
→ UC este oprit de 2 ori / transfer => μP pierde timp

B) Magistrală comună, controler DMA integrat




→ controlerul DMA este legat la mai multe porturi
→ fiecare transfer folosește magistrala o singură dată
→ DMA cu memoria
→ UC este oprit o singură dată / transfer

C) Magistrală de intrare/ieșire distinctă




→ fiecare transfer folosește magistrala comună o singură dată: DMA cu mem
→ UC este oprit o singură dată/transfer

# 46. Cum realizează un periferic un transfer DMA?

- Comunicarea UC – DMA – periferic:
  - Perifericul cere transfer de tip DMA activând intrarea DREQ (DMA Request) a controlerului DMA;
  - Controlerul DMA activează ieşirea sa HRQ spre intrarea HOLD a procesorului anunțându-l că are nevoie de magistrale;
  - Procesorul încheie ciclul maşină început, îşi trece magistralele şi semnalele de comandă în a 3 – a stare, respectiv în starea inactivă, şi activează ieşirea HLDA anunţând controlerul DMA că a cedat magistralele; cererea de pe intrarea HOLD trebuie să rămână activă pe toată durata transferului DMA;
  - Controlerul DMA activează ieşirea sa DACK comunicând perifericului că cererea sa DREQ a fost acceptată şi va începe transferul;
  - Controlerul DMA execută transferurile:
    - Memorie – port: pune adresa primului octet (cuvânt) pe magistrala de adrese, activează semnalul de comandă MEMR şi apoi IOW,
    - Port – memorie: activează semnalul de comandă IOR, pune adresa primului octet (cuvânt) pe magistrala de adrese şi apoi MEMW;
  - După încheierea transferurilor controlerul DMA dezactivează ieşirea HRQ anunţând procesorul că nu mai are nevoie de magistrale.

47. Ce este circuitul 82C37A? Prezentați caracteristicile sale.

Circuitul 82C37A este un controler DMA care gestionează transferul datelor între memorie și periferice fără intervenția constantă a procesorului.

- Dispune de 4 canale independente,
- Poate fi cascadat pentru a obţine oricâte canale,
- Control individual (activare/ dezactivare şi polaritate) asupra fiecărei cereri DREQ şi fiecărui răspuns DACK,
- Permite transfer memorie – memorie,
- Rată de transfer ridicată: până la 4 MO/ sec. sau până la 6.25 MO/ sec. în funcție de frecvența tactului;
- Asigură incrementarea sau decrementarea adresei,
- Se foloseşte cu un registru extern pentru jumătatea mai semnificativă a adresei,
- Poate lucra în mai multe moduri,
- Compatibil TTL/ CMOS.

# 48. Descrieți întreruperile externe.

I.e. sunt provocate de evenimente externe µP care cer atenție imediată din partea acestuia. Cel care cere întreruperea, trebuie să:

→ țină cererea activată până când µP răspunde.

→ să se identifice atunci când µP o cere

Pot exista cereri simultane din partea mai multor surse ⇒ prioritizare cu:

→ logică de prioritizare : | serială
| paralelă

→ PIC ( controler de întreruperi)

Prioritizarea poate fi | fixă
| programabilă
| rotativă

Există 2 tipuri de întreruperi externe:

A) mascabile → vin din partea periferiei sau a altui µP (în sisteme multiP)
→ mascarea se face prin program dezactivând sistemul de întreruperi
→ mascarea poate fi totală sau parțială ( se permite ca doar unele cereri să fie luate în considerare)

B) nemascabile → µP le ia în seamă întotdeauna
→ nu pot fi blocate decât prin hardware extern
→ rezervate pentru situații catastrofale
→ o cădere iminentă de tensiune
→ o eroare la memorie

Întreruperile sunt distinse prin
→ intrări diferite | INTR (mascabile)
| NMI (nemascabile)
→ coduri unice

Ele pot fi declanșate în 2 feluri:

A) active pe nivel (0 sau 1)
→ cererea trebuie să fie menținută activă până când este procesată
→ în rest, inactivă

B) active pe front ( ↑ sau ↓)
→ risc de pierdere al cererii dacă ∄ un mecanism de memorare
→ poate fi transformată în A) cu hardware extern ( bistabil)

49. Descrieți întreruperile interne.

Se găsesc la μP peste 8 biți și la toate μC. Se numesc și excepții la unele μP sau „traps".

Sunt provocate de cauze interne ale μP. La μC pot fi mascate, la μP nu.

2 tipuri de cauze (la μP)

1. Evenimente interne speciale
   - → generate înainte de execuția unei instrucțiuni: violare privilegii, cod ilegal, etc.
   - → generate după execuția unei instrucțiuni: divizare la 0, out of range, etc.

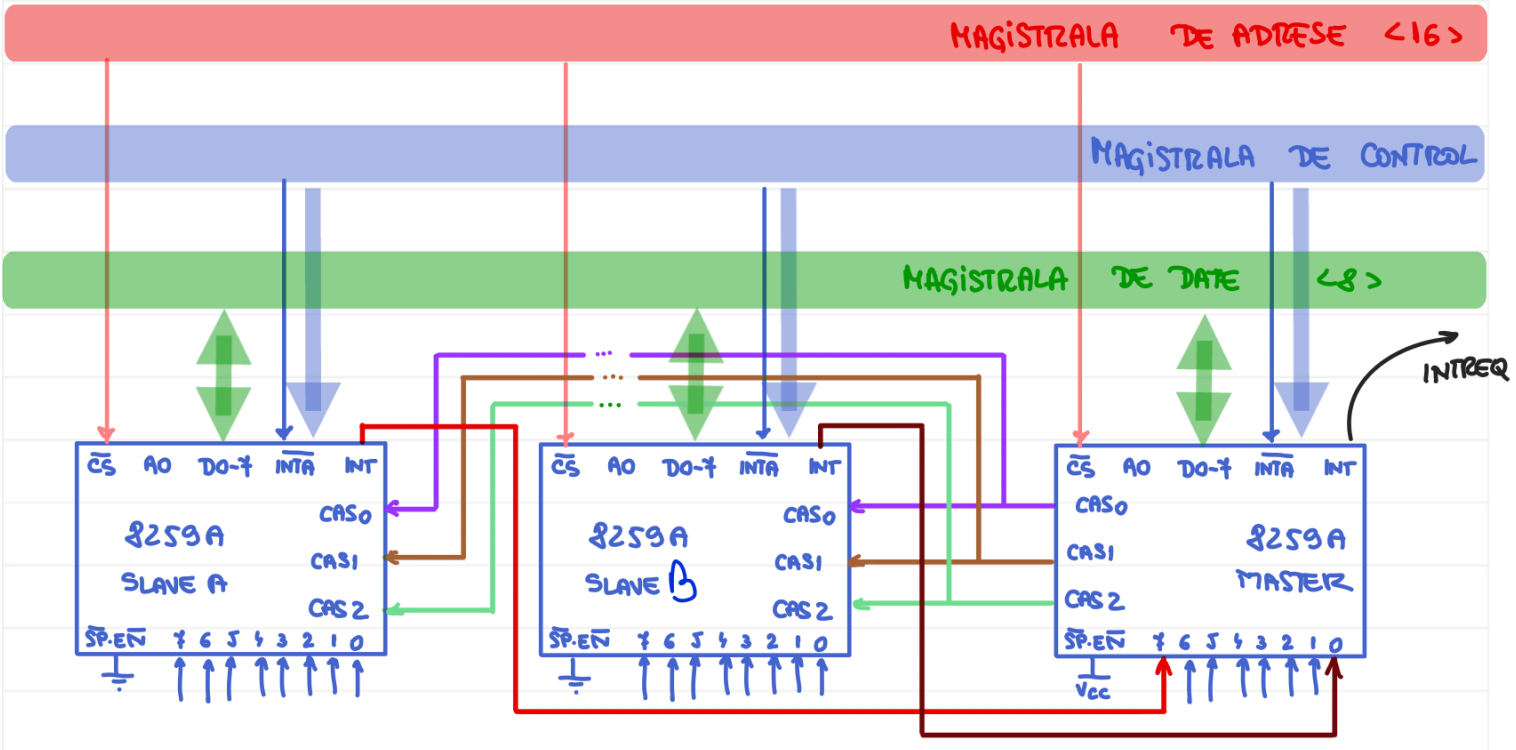
2. Instrucțiuni dedicate
   - → ∃ instr ⇔ generarea unei întreruperi (INT n la Intel)
   - → pot fi folosite pentru executarea de noi instrucțiuni

50. Ce este circuitul 8259? Prezentați caracteristicile sale.

Circuitul 8259A este un controler pentru întreruperi (PIC)

- Poate gestiona şi prioritiza 8 cereri de întrerupere; IR0-7
- Poate fi legat în cascadă cu alte 8 circuite similare asigurând astfel gestionarea şi prioritizarea a 64 cereri de întrerupere; → master - slave
- Posibilitate de mascare individuală a cererilor;
- Mai multe moduri de gestionare a întreruperilor:
  - Fully nested, IR0 max IR0 min
  - Rotating priority, acces egal
  - Special mask şi
  - Poll.
- Necesită 2 - 4 cuvinte de iniţializare (ICW1 – 4) şi 4 cuvinte de operare (OCW1 – 4);
- Generează vectorul de întrerupere asociat cererii luată în considerare de procesor.

# 51. Cum se face și la ce folosește legarea în cascadă a circuitelor 8259?




Legarea în cascadă folosește la depășirea limitei fizice de 8 întreruperi a unui singur cip.

Un circuit 8259 - master se poate conecta cu până la 8 circuite 8259 - slave putând gestiona 64 de cereri de întreruperi externe.

Circuitul are 3 terminale pt cascadare: CAS0, CAS1, CAS2 → conectează circuitele în paralel, iar master spune care slave trb. să se activeze.

Ieșirea de întrerupere INT de la fiecare slave este legată la una din cererile de întrerupere ale masterului (IR0/IR7)

Ieșirea -II- a masterului merge la µP

52. Justificați existența unui circuit de memorare (bistabil, registru) în schema de comandă a unui led prin program.

Existența unui circuit de memorare este necesară deoarece LED-ul are nevoie să primească un semnal de comandă stabil pentru a rămâne aprins sau stins, în timp ce datele de pe magistrala procesorului sunt valide doar pentru un timp foarte scurt.

Pe magistrala de date, informația se schimbă continuu, iar ochiul uman nu ar putea să sesizeze stingerea sau aprinderea LED-ului, fenomenul întâmplându-se prea rapid fără o memorare a stării.

**1. Conectarea Hardware:**

- Conectarea LED-urilor la unitatea centrală (UC) se face prin intermediul unui **port de ieșire**.
- Acest port de ieșire **"poate fi un bistabil sau un registru"**. De exemplu, schemele prezentate utilizează un bistabil (ex: 7474) sau un registru (ex: 74LS373).

**2. Funcționare și Comandă:**

- Procesorul trimite comanda printr-o secvență de instrucțiuni de tipul `MOV AL, valoare` urmată de `OUT port, AL`.
- Instrucțiunea `OUT` pune datele pe magistrală doar temporar.
- Circuitul de memorare (bistabilul) preia această valoare logică ("0" sau "1") și o menține (o zăvorăște) la ieșire, asigurând curentul necesar LED-ului (`IOL` sau `IOH`) continuu, până la următoarea comandă de actualizare a portului.

**3. Nivele logice:**

- În funcție de tehnologia circuitului de memorare și conectare, LED-ul poate fi comandat să lumineze pe:
  - **Nivelul "0" logic:** Specific circuitelor **TTL**, unde curentul absorbit (`IOL`) este mai mare.
  - **Nivelul "1" logic:** Specific circuitelor **HCT**.

53. Este necesar un bistabil pentru citirea stării unui comutator prin program?

In general, nu este nevoie de un bistabil pentru citirea starii unui comutator prin program, dar poate fi necesar pentru tratarea fenomenelor tranzitorii (vibratiile).

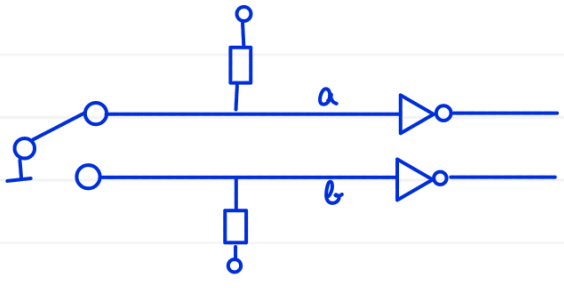
**1. Citirea standard (Fără bistabil):**

- Schemele de conectare a comutatoarelor prezentate în curs utilizează **tampoane (buffere)** sau porți cu trei stări (ex: 74LS244, reprezentat triunghiular în scheme) activate de un decodificator.
- Starea comutatorului este citită direct prin instrucțiuni de tip `IN AL, port` urmate de o mascare (`AND`) pentru a izola bitul dorit și o instrucțiune de salt condiționat (`JZ` sau `JNZ`).

**2. Tratarea vibrațiilor (Cu bistabil):**

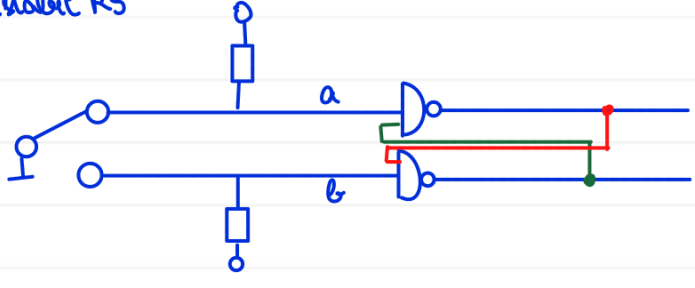
- În cazul comutatoarelor mecanice apare **"problema vibrațiilor"** (contactele oscilează rapid între închis și deschis la acționare).
- Pentru eliminarea efectului acestor vibrații, cursul menționează ca soluție hardware **"conectarea unui bistabil RS asincron"**.
- Aceasta este caracterizată drept o **"soluție corectă, performantă dar scumpă"**. Alte soluții menționate sunt cele software (întârziere de aprox. 5 ms) sau utilizarea condensatoarelor (nerecomandată în producție de serie).

**Concluzie:** Bistabilul nu este strict necesar pentru mecanismul de citire (unde se folosesc buffere), dar este o componentă opțională utilizată pentru eliminarea hardware a vibrațiilor contactelor mecanice.



cu condensatoare nu e recomandat

bistabil RS



## 54. Care este diferența între arhitectura și microarhitectura unui microprocesor?

Diferența constă în nivelul de abstracție și vizibilitate pentru programator versus implementarea fizică:

Arhitectura: Reprezintă definiția setului de instrucțiuni și compatibilitatea acestuia. Include elementele accesibile și vizibile programatorului, cum ar fi setul de instrucțiuni, registrele și structura datelor în memorie.

Microarhitectura : Reprezintă implementarea efectivă a arhitecturii pe suportul fizic (hardware). Aceasta este "motorul" din spatele arhitecturii și este îmbunătățită continuu de la o generație de procesoare la alta (ex: P5, P6, NetBurst), menținând însă compatibilitatea cu arhitectura de nivel înalt (ex: IA-32)

In plus, o arhitectura poate avea mai multe microarhitecturi, pe care sunt construite unul sau mai multe tipuri de microprocesoare.

## 55. Prin ce diferă structura Von Neumann de structura Harvard? Ce avantaje și dezavantaje au cele 2 structuri?

Diferența fundamentală constă în modul de organizare și accesare a memoriei pentru cod (instrucțiuni) și date.

Von Neumann:

- Codul și datele sunt reprezentate în memorie la fel și sunt accesate la fel
- Se folosesc aceleași magistrale
- Întrucât toate accesele se fac la aceeași memorie și memoria este mai lentă ca procesorul (la toate nivelele tehnologice), aceasta va frâna execuția programului

Avantaj :

- simplitate (datorat faptului ca se folosesc aceleasi magistrale)
  Dezavantaj:
- Frecventa de lucru redusa, timp mare de acces

Harvard:

- Codul și datele sunt păstrate în memorii fizice diferite (există două spații de adrese distincte)
- Accesul la cod și date se face prin magistrale diferite

Avantaj:

- Viteză mare de execuție, permițând accesul simultan la instrucțiuni și date

Dezavantaj:

- Scheme hardware mai complicate (magistrale dublate)

56. Dați exemple de arhitecturi ale setului de instrucțiuni ale microprocesoarelor. Caracteristici.

- CISC (Complex Instruction Set Computer)
  - Instrucțiuni complexe, lungi, cu multe moduri de adresare și cu durată mare de execuție;
  - Programarea este mai simplă, compilatorul este complex, viteză redusă;
  - Exemple: arhitectura Intel x86, Motorola 68k;
- RISC (Reduced Instruction Set Computer)
  - Instrucțiuni simple, scurte, cu durată mică de execuție dar și cu efect redus; → necazul programatorului: are mai mult de lucru, mai mult cod de scris
  - Număr redus de moduri de adresare, instrucțiuni pe 1 sau puține cicluri, compilatoare mai simple, viteză mai mare;
  - Exemple: PowerPC, ARM, SPARC;
- VLIW (Very Long Instruction Word)
  - Instrucțiuni lungi, pe mai multe cuvinte;
  - Dacă procesorul este multicore, se pot grupa mai multe instrucțiuni/core, pentru a rezulta o instrucțiune lungă;

57. Descrieți evoluția soluțiilor implementate în microprocesoare pentru execuția instrucțiunilor (fără hyperthreading).

- pentru execuția unei instrucțiuni, μP parcurge un ciclu al instrucțiunilor
- un ciclu al instrucțiunii este divizat în cicluri mașină
- un ciclu mașină este divizat în stări
- stare = perioadă de tact
- fiecare μP știe să execute un număr limitat de cicluri mașină
- prin combinarea lor ⇒ ciclurile de instrucțiuni

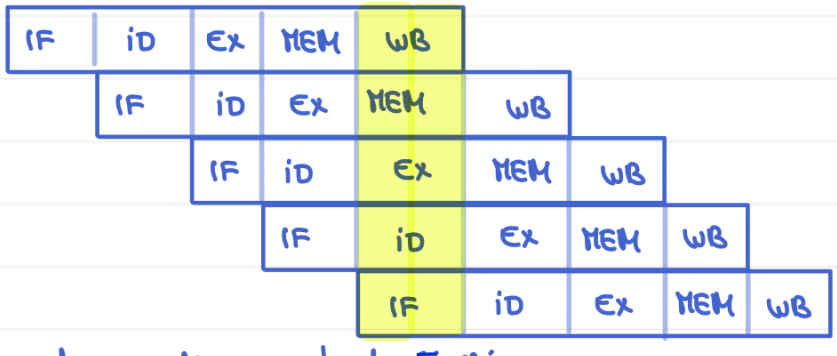
## ① μP cu EXECUȚIE SECVENȚIALĂ:

- considerăm un exemplu în care un ciclu instrucțiune are 5 cicluri mașină
- urmărim felul în care se execută aceste instrucțiuni în μP cu execuție secvențială:

- dezavantaje: viteză mică, hardware ineficient folosit
- avantaj: simplitate „totul e liniar și putem determina duratele instrucțiunilor

## ② μP cu BANDĂ DE ASAMBLARE (pipelining)

- banda de asamblare: împărțită în posturi
- în teorie, un număr mai mare de posturi ⇒ eficiență sporită




- rata de execuție crește de 5 ori
- timp mai redus, de la 5×n la n+4
- o problemă ar fi hazardele, care rup secvențierea codului
  ⇒ instrucțiunile care au intrat eronat în bandă vor determina golirea benzii
  * cel mai rău: instrucțiunile condiționate care rup

• a doua problemă: latența: posturile nu pot funcționa toate cu aceeași viteză ⇒
⇒ celelalte posturi, cu latență mică, vor aștepta după posturile cu latență mare

→ hardware complex

## ③ μP cu SUPERPIPELINE

• față de μP cu bandă de asamblare, postul cu latență mare a fost rupt în posturi cu latența egală cu a celorlalte posturi ⇒ latența scade, dar se complică hardware-ul

## ④ μP SUPERSCALARE

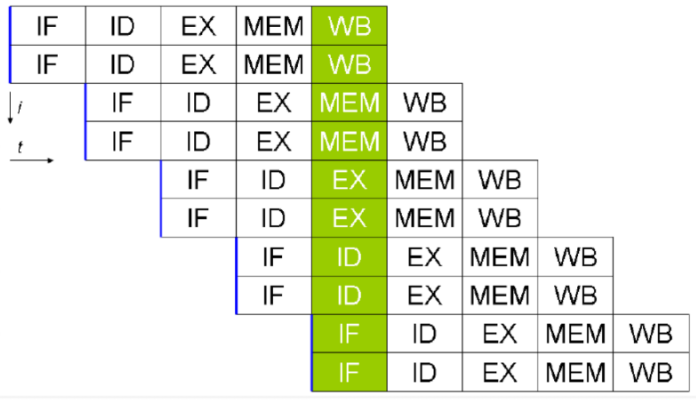
• dau impresia de execuție în paralel, fiind un asemenea procesor nu are mai multe nuclee în el

• doar un singur nucleu, DAR cu mai multe resurse, mai multe linii de execuție

• nucleul se multiplexează rapid, de la o linie de execuție la alta, dând impresia de execuție în paralel

• dezavantaj: gestiunea liniilor de execuție cere mai multe acces la memorie

• avantaj: câștig de timp clar : n+1 cicluri de tact




## ⑤ HYPERTHREADING: superscalaritatea Intel

## ⑥ PROCESOARE VLIW (Very Long Instruction Word): fiecare nucleu primește o instrucțiune de executat la fiecare cuvânt; planificarea este făcută de compilator

## ⑦ PROCESOARE VECTORIALE:

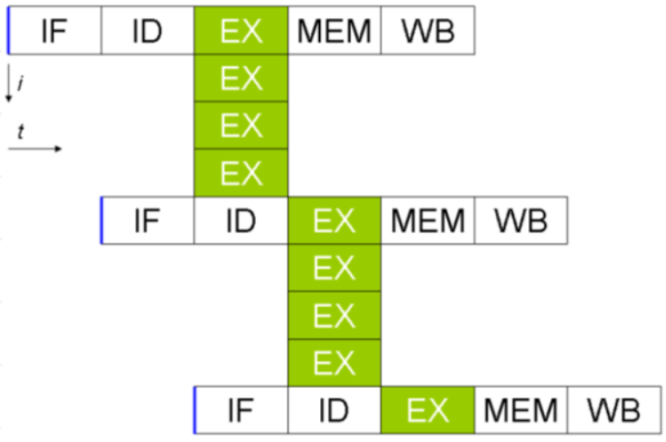
• clasificarea lui Flynn: S/M I S/M D

S/M: single / multiple

I: Instruction Stream

D: Data Stream

- mai multe nuclee
- execuție aceeași la fiecare nucleu
- seturile de date diferă, aceeași operație
- utile în aplicațiile audio - video
- se pp. 7 a mai multe UAL




(8) μP VECTORIALE cu EXECUȚIE SERIALĂ: partea de hardware se reduce, este suficientă una
: pierdem timp
→ un singur UAL

(9) μP MULTICORE : μP cu mai multe nuclee independente
: pot fi simetrice și asimetrice

(10) μP MANYCORE : mai multe nuclee ca procesoarele multicore
: optimizate pentru procesare masiv paralelă

(11) SoC (system-on-chip) & NoC (network-on-chip)
- mai multe rețele legate prin rețele de interconectare
- mai complexe decât μC
- mai aproape de μP după al scopului pentru care a fost construit

58. Care este cauza limitării eficienței benzii de asamblare? Prezentați soluții pentru creșterea eficienței de asamblare.

Cauza limitarii este faptul ca eficienta scade dramatic la apariția instrucțiunilor care rup secvențialitatea programului, în special **instrucțiunile de salt condiționat sau apel condiționat**. Acestea introduc "hazardul de control", deoarece procesorul nu știe ce instrucțiune să aducă în bandă până nu se evaluează condiția. Practic, latenta nu este egala pe toate posturile, iar posturile mai rapide vor trebui sa astepte dupa postul cel mai lent. O solutie ar fi ca posturile lente sa fie rupte in mai multe posturi cu latenta cea mai mica.

Soluții :

- Execuția speculativă: se ghicește ramura unui salt/apel condiționat care va fi continuată; în gazul ghicirii eronate, banda trebuie golită; în cazul ghicirii corecte, nu se pierde timp;
- Întârzierea execuției sau furtul de ciclu: după instrucțiunea de salt/apel condiționat se plasează o instrucțiune care va fi executată indiferent de ramura care va continua; dacă nu sunt dependențe de date, această instrucțiune poate fi NOP; se câștigă timp pentru ca instrucțiunea de salt/apel condiționat să stabilească ramura pe care va continua;

Branch delay Not

- Predicția ramurii: se ghicește ramura care se va executa dar nu aleator ci bazat pe anumite considerente, de obicei istoria salturilor;
  - Ex.: execuția unei bucle; o buclă este executată de mai multe ori înaintea terminării ei, însemnând că au loc mai multe ramificări în o direcție și doar o singură ramificare în cealaltă direcție; fie secvența în pseudocod HLL:

predict taken

# 59. Ce înseamnă superscalaritate?

Superscalaritatea este o arhitectură în care un microprocesor cu **un singur nucleu** dispune de **mai multe resurse de execuție** duplicate (multiple Unități Aritmetice și Logice - UAL, multiple unități de virgulă flotantă - FPU etc.). Acest lucru permite procesorului să execute **simultan** mai multe instrucțiuni independente în cadrul aceluiași ciclu de ceas.

---

Raspuns mai lung:

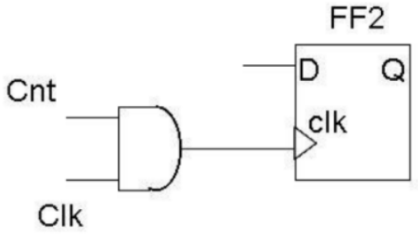
Superscalaritatea este o arhitectură de procesor care permite executarea simultană a mai multor instrucțiuni într-un singur ciclu de ceas. Aceasta înseamnă că un procesor superscalar poate emite și executa mai multe instrucțiuni în paralel, spre deosebire de un procesor scalar, care execută câte o singură instrucțiune într-un ciclu de ceas.

Un procesor superscalar are mai multe unități de execuție (de exemplu, unități aritmetico-logice sau unități de încărcare/stocare) care pot prelua și executa instrucțiuni diferite în același timp, ceea ce îmbunătățește performanța procesorului prin reducerea timpului de execuție pentru programele care pot fi paralelizate.

Superscalaritatea nu înseamnă că toate instrucțiunile pot fi executate simultan, deoarece există limitări legate de dependențele între instrucțiuni (de exemplu, o instrucțiune care depinde de rezultatul altei instrucțiuni) și de capacitatea procesorului de a organiza și gestiona instrucțiunile într-un mod eficient. Totuși, arhitectura superscalară poate duce la o îmbunătățire semnificativă a performanței procesorului, în special în aplicațiile care pot beneficia de paralelism.

60. Ce dezavantaje are creșterea frecvenței tactului la un microprocesor? De ce?

- Creșterea frecvenței este limitată de soluțiile pentru scăderea puterii;
  - Clock-gating:



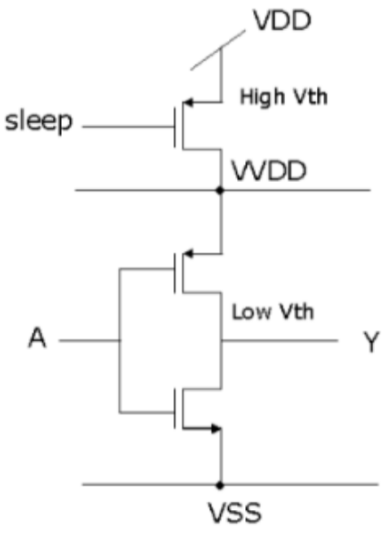

    - Există o validare a tactului care permite încărcarea informației în registre doar dacă datele s-au modificat; este economisită putere dar și limitată frecvența;

Principalul dezavantaj asociat cresterii frecventei este legat de consumul de putere. Solutiile gasite pentru scaderea puterii(clock-gating sau power-gating) introduc si ele limitari asupra frecventei maxime.

- **Disiparea căldurii (Thermal Wall):**
  - Puterea consumată se transformă în căldură. O frecvență prea mare duce la o încălzire excesivă pe o suprafață foarte mică de siliciu, care nu mai poate fi răcită eficient (risc de distrugere a cipului).
- **Problema "Memory Wall" (Zidul Memoriei):**
  - Chiar dacă creștem frecvența procesorului, memoria RAM nu poate ține pasul. Procesorul va rula extrem de rapid, dar va sta degeaba așteptând datele din memoria lentă. Creșterea frecvenței devine inutilă fără o memorie la fel de rapidă.
- **Integritatea semnalului:**
  - La frecvențe foarte mari (GHz), semnalele electrice suferă fenomene parazite (interferențe, întârzieri de propagare pe trasee lungi), ceea ce poate duce la erori de calcul sau instabilitate.

# 61. Ce înseamnă power-gating? La ce este util? Ce dezavantaje are?

- Power-gating:




  - Tranzistorul de sus poate opri alimentarea etajului final, economisind putere dar introduce timpul necesar tranziției sale, limitând frecvența;

Este o tehnică de reducere a consumului care implică utilizarea unui tranzistor dedicat (controlat de un semnal "sleep") pentru a întrerupe alimentarea (VDD) unui etaj sau bloc funcțional al procesorului atunci când acesta nu este utilizat

**2. La ce este util:**

- **Reducerea curentului de scurgere (Leakage):** Spre deosebire de *clock-gating* (care oprește doar ceasul și reduce puterea dinamică), power-gating elimină și consumul static (pierderile de curent care apar chiar și când tranzistoarele nu comută).
- Este esențial pentru dispozitivele mobile, permițând oprirea unor zone mari din cip (ex: un nucleu întreg, unitatea grafică sau modemul) pentru a economisi bateria în standby.

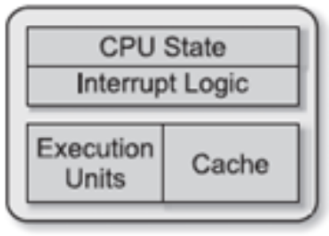
---

**3. Dezavantaje:**

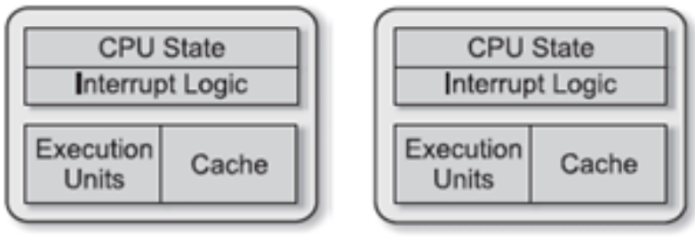
- **Latența de revenire (Wake-up Latency):** Procesul de realimentare și stabilizare a tensiunii durează, ceea ce introduce întârzieri la repornirea blocului funcțional.
- **Pierderea stării interne:** Odată tăiată alimentarea, datele din registrele acelui bloc se pierd. Este necesară salvarea contextului (în memorie) înainte de oprire și restaurarea lui la pornire, ceea ce consumă timp și energie.
- **Limitarea frecvenței (Conform slide-ului anterior):** Tranzistoarele adăugate pentru comutare introduc o rezistență suplimentară pe liniile de alimentare, ceea ce poate limita frecvența maximă de operare a circuitului atunci când este activ.

62. Care sunt diferențele principale între microprocesoarele unicore, multimicroprocesoare, microprocesoarele multitasking și cele multicore?

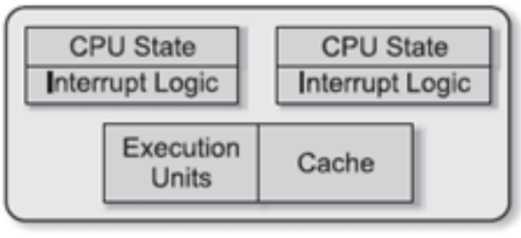
- Microprocesoare unicore
  - Execuție secvențială a instrucțiunilor;
  - Facilități: pipeline, out-of-order execution (instruction level parallelilsm)
    - Performanța limitată de interdependențele între instrucțiuni;
    - Performanța pipeline-ului limitată de instrucțiunile care modifică secvențialitatea;
- Multimicroprocesoare: sisteme cu mai multe microprocesoare + circuite, cu acces comun la resurse;
  - Problemă: accesul la resursele comune;
- Microprocesoare multitasking (multithreading)
  - Un unic microprocesor fizic este văzut de software ca mai multe microprocesoare logice;
  - Fiecare microprocesor logic execută un task;
  - Problemă: comutarea taskurilor;
- Microprocesoare multicore
  - Mai multe microprocesoare fizice (core (nucleu)) execută fiecare câte un task;



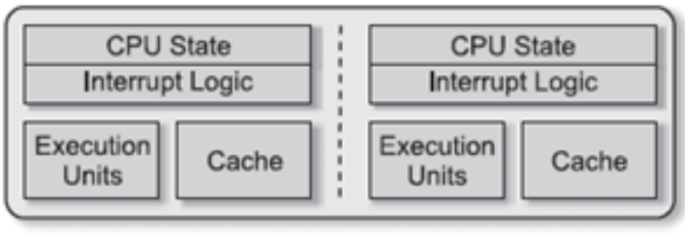

A) Single Core



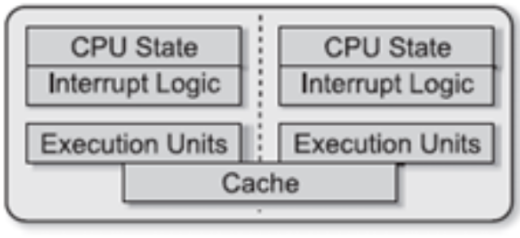

B) Multiprocessor



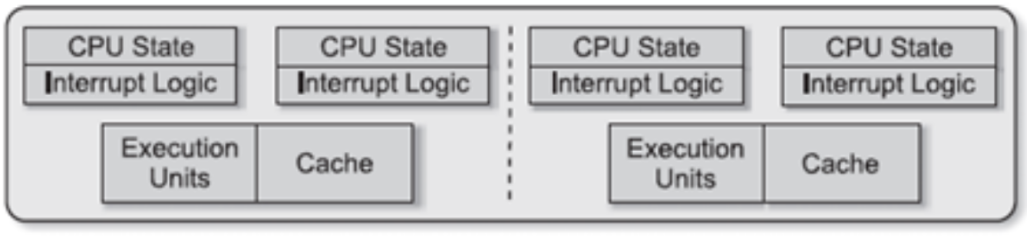

C) Hyper-Threading Technology




D) Multi-core

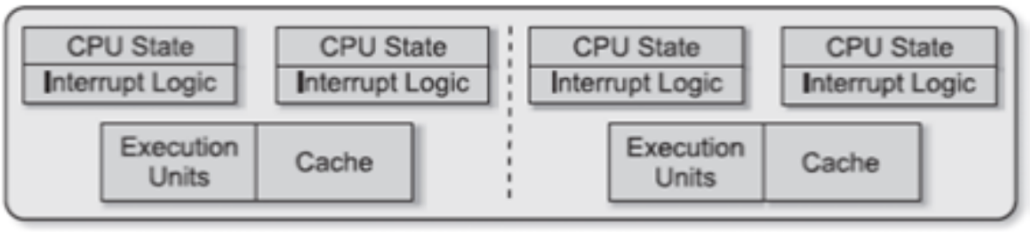



E) Multi-core with Shared Cache




F) Multi-core with Hyper-Threading Technology

# 63. Prezentați schema bloc a unui procesor multicore cu hyperthreading.




F) Multi-core with Hyper-Threading Technology

Structura unui astfel de microprocesor combină arhitectura multicore cu cea multithreading. Schema bloc prezintă următoarele caracteristici:

• **Nivelul Fizic (Cipul):** Cipul este divizat în două (sau mai multe) regiuni mari, reprezentând **Nucleele fizice** separate.

• **Interiorul unui Nucleu:** În cadrul fiecărui nucleu fizic, resursele sunt organizate astfel:

◦ **Resurse Duplicate (Logice):** Există două blocuri de **CPU State** (starea procesorului - registre) și **Interrupt Logic** (logica de întreruperi). Aceasta permite sistemului de operare să vadă două procesoare logice pe un singur nucleu fizic.

---

◦ **Resurse Partajate (Fizice):** Cele două procesoare logice din cadrul aceluiași nucleu împart un singur set de **Execution Units** (Unități de Execuție - UAL, FPU) și memoria **Cache** proprie nucleului.

• **Separarea între Nuclee:** Nucleele sunt complet separate între ele (linia punctată din diagramă), având fiecare propriile resurse de execuție și cache, spre deosebire de partajarea din interiorul nucleului.

Această structură permite ca, pe un procesor fizic cu 2 nuclee, sistemul de operare să vadă și să utilizeze 4 procesoare logice (thread-uri).

# 64. Ce arată legea lui Amdahl?

G. Amdahl a considerat ca orice program paralel are:

- o componenta secventiala care nu poate fi paralelizata(S)
- O parte complet paralelizabila (P)

Timpul de executie al programului pe un procesor unicore va fi S + P
Timpul de executie pe un procesor cu n nuclee va fi S + P / n

Formula generală pentru calculul accelerării (Speedup) în funcție de numărul de nuclee este:

$$\text{Speedup}(n) = \frac{1}{S + \frac{P}{n}} < \frac{1}{S}$$

$$Speedup\ (n) = (S + P)/(S + P/n) = 1/((1 - P) + P/n) = 1/(S + P/n) \leq 1/S$$

adică creșterea de viteză este limitată de porțiunea din cod neparalelizată, indiferent de numărul de nuclee;

**Unde:**

- **S** = componenta secvențială (neparalelizabilă).
- **P** = componenta paralelizabilă (unde $P = 1 - S$).
- **n** = numărul de nuclee de procesare.

- Creşterea numărului de nuclee nu duce automat la creşterea performanţelor; FA este limitat de inversul fracţiunii din program care trebuie executată secvenţial;

factorul de accelerare

- Dacă *S = 0,1* atunci *FA ≤ 10* indiferent de numărul de procesoare;
- La limită, dacă programul este complet secvenţial, adică *P = 0*, atunci *FA ≤ 1* adică durata de execuţie a programului pe un sistem multicore va fi egală sau mai mare decât pe un sistem unicore; degradarea acestui timp apare datorită comunicării între nuclee, în cazul sistemului multicore;
- Dacă jumătate din program este executat de un procesor cu 2 nuclee atunci factorul de accelerare va fi:

*Speedup (2) = 1/(0,5 + 0,5/2) = 1/0,75 = 1,33*

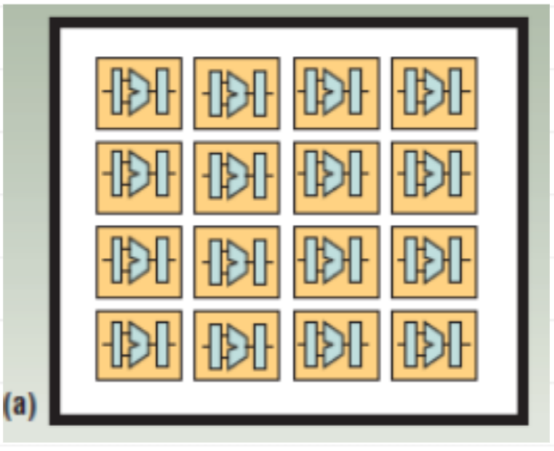
adică un câștig de performanță de 33%; la 8 nuclee creșterea este de 78%

- Din legea lui Amdahl rezultă că FA este limitat superior de o valoare care nu depinde de numărul de nuclee sau de arhitectura procesorului;
- Dacă *n* → infinit, atunci rezultă aceeași concluzie:

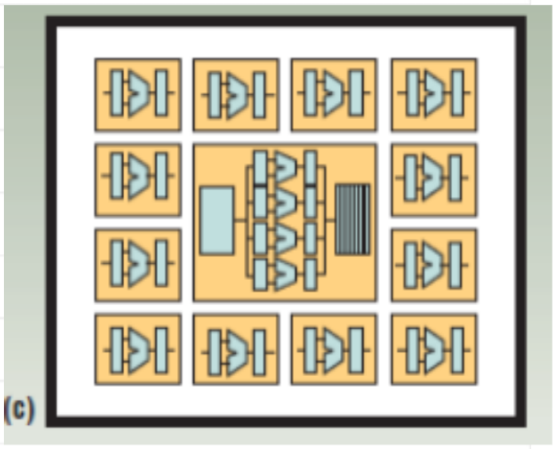
*Speedup = 1/(1 – P) = 1/S*

- Corolar: este mai eficientă creșterea porțiunii din cod paralelizată decât creșterea numărului nucleelor; ex.: cod paralelizat în raport de 30%, rulând pe un procesor cu 2 nuclee; dublând numărul nucleelor scade timpul de rulare de la 85 unități la 77,5 unități; dublând porțiunea din cod paralelizată, scade timpul de rulare de la 85 unități la 70 unități;

65. Prezentați structura unui microprocesor multicore simetric și a unuia asimetric.




- a: multicore simetric cu 16 nuclee cu 1 resursă;




- c: multicore asimetric cu 12 nuclee cu 1 resursă și 1 nucleu cu 4 resurse;

**Definiție de bază:** Procesoarele simetrice au toate nucleele la fel, iar cele asimetrice au nuclee diferite. Un nucleu poate avea mai multe resurse.

### 1. Multiprocesarea Simetrică (SMP - Symmetric Multi-Processing)

Aceasta este structura dominantă în sistemele moderne (PC-uri, servere, smartphone-uri).

- **Structură:** Toate nucleele (procesoarele) sunt identice din punct de vedere arhitectural.
- **Acces la Resurse:** Toate nucleele au acces egal și partajat la resursele sistemului, cum ar fi memoria principală și dispozitivele de intrare/ieșire (I/O).
- **Sistem de Operare:** Rulează o singură copie a sistemului de operare care gestionează toate nucleele simultan.
- **Funcționare:** Orice task (proces) poate fi executat pe oricare dintre nuclee, sistemul de operare distribuind dinamic încărcarea pentru a echilibra efortul.

### 2. Multiprocesarea Asimetrică (AMP - Asymmetric Multi-Processing)

Această structură este folosită adesea în sisteme specializate sau în arhitecturi vechi, bazându-se pe o ierarhie strictă.

- **Structură:** Nucleele pot fi diferite sau au roluri prestabilite distincte. Arhitectura este de tip **Master-Slave** (Stăpân-Sclav).
- **Rolul Master:** Un singur procesor (Master) controlează întregul sistem. El rulează sistemul de operare, gestionează resursele și atribuie sarcini celorlalte procesoare.
- **Rolul Slave:** Celelalte procesoare (Slave) execută doar sarcinile specifice atribuite de Master sau au funcții predefinite dedicate (ex: procesare grafică, procesare semnal).
- **Acces la Resurse:** De obicei, procesorul Master gestionează accesul la memorie și I/O pentru întregul sistem.

# 66. Prezentați caracteristici și cerințe ale sistemelor încorporate

Avem 6 definitii pentru Sistemele incorporate:

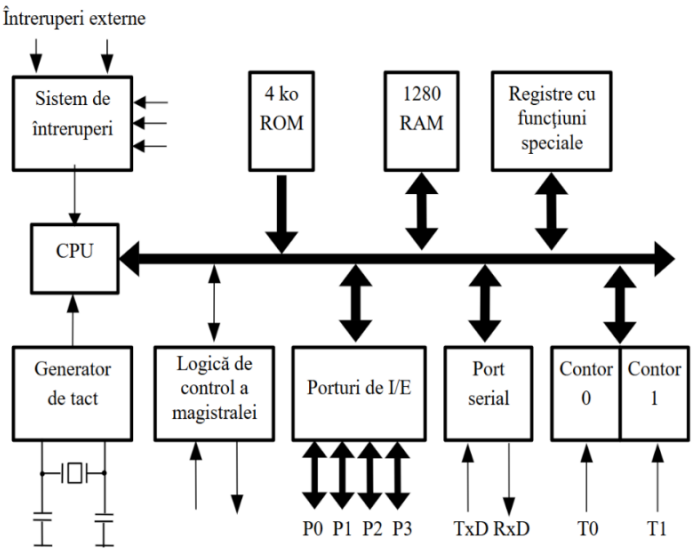
- Definiţia 1: Un SI este un sistem integrând hardware şi software şi proiectat pentru o anumită funcţionalitate;
- Definiţia 2: Un SI este in sistem de calcul cu scop predefinit inclus într – un dispozitiv pe care îl conduce;
- Definiţia 3: Un SI este un sistem de calcul cu cerinţe specifice; spre deosebire de calculatorul de uz general, SI execută sarcini predefinite;
- Definiţia 4: Un SI este un sistem de procesare a informaţiei, parte a unui sistem mai mare sau a unui dispozitiv;
- Definiţia 5: Un SI este este o combinaţie de hardware şi software cu programare şi facilităţi fixe, proiectat pentru un tip de aplicaţii;
- Definiţia 6: Un SI este o combinaţie de hardware şi software şi, posibil, elemente mecanice sau alte elemente, proiectat pentru a realiza o funcţie dedicate; în unele cazuri, SI sunt părţi ale unui sistem mai mare sau produs, ca de exemplu sistemul ABS ("Antilock Bracking System") dintr – un automobil;

Aceste sisteme sunt destinate unei aplicații sau familii de aplicații și pot conține un microcalculator dar, în general, sunt sisteme de sine-stătătoare incluse în dispozitivul pe care îl conduc. Un Sistem Încorporat (SI) trebuie să lucreze în condiții mai restrictive decât un calculator de uz general.

**Caracteristici și cerințe:**

- **Conectare la mediul exterior:** Monitorizează mediul prin intermediul **senzorilor** și îl comandă prin intermediul **actuatorilor**.
  - *Notă:* Un actuator poate fi definit ca un dispozitiv care convertește valori numerice în efecte fizice.
- **Funcționare reactivă:** Un sistem reactiv este în continuă interacțiune cu mediul înconjurător și execută sarcinile la rata determinată de mediu.
  - Poate fi gândit ca fiind într-o anumită stare, așteptând o intrare; pentru fiecare intrare execută una sau mai multe operații și generează o ieșire.
  - Un asemenea sistem poate fi modelat foarte bine de un automat de stări.
- **Funcționare în timp real:** Timpul devine un parametru al execuției operațiilor. Există două tipuri de constrângeri:
  - **Hard (stricte):** Nerespectarea lor produce efecte grave, uneori dezastruoase.
  - **Soft (ușoare):** Nerespectarea lor produce doar efecte negative (degradarea performanței).
- **Eficiența:** Un SI trebuie să fie eficient, aspect evaluat prin următoarele metrici:
  - **Consumul de energie:** Trebuie minimizat, mai ales în cazul SI mobile.
  - **Dimensiunea codului:** Cod mare → memorie de program mare (posibil externă) → circuite suplimentare → dimensiune mai mare a SI, consum mai mare, fiabilitate mai mică.
  - **Hardware minim:** Execuție implicând un minim de circuite.
  - **Fizic:** Greutate și dimensiuni mici.
  - **Economic:** Cost redus.
- **Funcționare în medii grele:** Trebuie să reziste la căldură excesivă, vibrații, coroziune sau fluctuații ale tensiunii de alimentare.
- **Dependabilitate:** Este foarte importantă datorită conexiunii cu mediul exterior și cuprinde următoarele aspecte:
  - **Fiabilitate:** Probabilitatea ca un sistem să nu se defecteze.
  - **Mentenabilitate:** Probabilitatea ca o defecțiune să poată fi reparată într-un timp anumit.
  - **Siguranță:** Probabilitatea ca o defecțiune să nu cauzeze efecte catastrofale.
  - **Disponibilitate:** Probabilitatea ca un sistem să fie funcțional și accesibil atunci când este nevoie de el.

# 67. Prezentați structura internă a microcontrolerului 8051.




## Structura și Resursele Microcontrolerului 8051

Microcontrolerul 8051 este un circuit pe 8 biți realizat în tehnologie CMOS, bazat pe o **Arhitectură Harvard** (ceea ce înseamnă că spațiul de memorie pentru program și cel pentru date sunt separate fizic și logic).

Structura internă cuprinde următoarele blocuri funcționale:

### 1. Unitate Centrală (CPU)

Este "creierul" microcontrolerului și include:

- **Unitatea Aritmetică și Logică (UAL):** Execută operații matematice și logice pe 8 biți.
- **Procesorul Boolean:** Un modul specializat care permite manipularea directă a variabilelor de 1 bit (setare, ștergere, complementare bit).
- **Registre specifice:**
  - **Acumulatorul (A):** Registrul principal de lucru.
  - **Registrul B:** Folosit împreună cu Acumulatorul pentru operații de **înmulțire și împărțire**.
  - **DPTR (Data Pointer):** Un registru de **16 biți** folosit pentru adresarea memoriei externe de date.

### 2. Organizarea Memoriei

Microcontrolerul dispune de memorie internă, dar și de capabilități de extindere externă:

- **Memorie Internă:**
  - **4 Ko ROM:** Pentru stocarea programului.
  - **128 octeți RAM:** Pentru date. Acești 128 de octeți sunt împărțiți în:
    - *4 Bănci de registre (R0-R7).*
    - *Zona adresabilă la nivel de bit (20H – 2FH).*
    - *Zona RAM de uz general (Scratchpad).*
- **Memorie Externă (Extensibilitate):**
  - Poate adresa până la **64 Ko memorie de Program** (externă).
  - Poate adresa până la **64 Ko memorie de Date** (externă).

### 3. Porturi de Intrare/Ieșire (I/E)

Dispune de 4 porturi bidirecționale pe 8 biți ( `P0` , `P1` , `P2` , `P3` ):

- Pot fi utilizate ca porturi de uz general sau cu funcțiuni alternative.
- **Notă:** Portul `P0` este de tip **Open-Drain** și necesită rezistențe de pull-up externe când este folosit ca port I/O simplu.

### 4. Temporizare și Numărare

- Conține **2 contoare/temporizatoare independente pe 16 biți** ( `T0` , `T1` ).
- Acestea pot funcționa ca Timer (ceas intern) sau Counter (impulsuri externe).

### 5. Comunicație Serială

- Include un port serial universal (**UART**) full-duplex.
- Utilizează liniile `TxD` și `RxD` pentru comunicarea asincronă.

### 6. Sistemul de Întreruperi

Gestionează evenimentele asincrone prin **5 surse de întreruperi**:

- 2 Întreruperi externe ( `INT0` , `INT1` ).
- 2 Întreruperi de la temporizatoare ( `T0` , `T1` ).
- 1 Întrerupere de la portul serial.
- Sistemul suportă **2 nivele de prioritate** programabile.

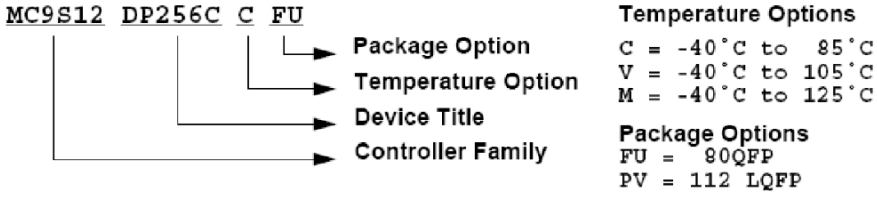
### 7. Generatorul de Tact și Control

- **Oscilator intern:** Circuitul funcționează pe baza unui cristal de cuarț extern.
- **Ciclul Mașină:** Unitatea de bază de timp este egală cu **12 perioade de oscilator**.
- **Logica de control:** Include magistralele și registrele cu funcțiuni speciale (SFR).

# 68. Prezentați caracteristicile microcontrolerului MC9S12DJ256.

- Exemplu: MC9S12DJ256;
- Include unitate centrală de tip HCS 12, compatibilă cu HC 12 şi 11;
- Include interfaţă pentru magistrală externă multiplexată, modul de breackpoint, modul pentru depanare;
- Include modul de tact complex: oscilator cu consum mic, PLL, monitor de tact, COP watchdog;
- Porturi pe 8 şi 4 biţi, cu facilităţi de întrerupere;
- Contor pe 16 biţi cu 8 canale de captare/ comparare; contor acumulator (2 pe 8 biţi sau 1 pe 16 biţi);
- 8 canale PWM;
- 2 interfeţe seriale asincrone şi 3 interfeţe seriale sincrone;
- Interfaţă serială pentru aplicaţii automotive şi interfaţă Inter IC (IIC);

- Operare la 50 MHz;
- Breackpoints hardware;
- CAN cu intrări la 5 V;
- Linii de I/E cu operare la 5V; care este câștigul?
- Moduri de operare utilizator, speciale şi cu consum redus;
- Deşi magistralele sale sunt pe 16 biţi, poate lucra cu magistrală externă de date de 8 biţi;
- Includerea unui PLL permite adaptarea consumului şi performanţelor cu cerinţele aplicaţiei;
- Semnificaţia codului circuitului:




Pe scurt:
Microcontrolerul MC9S12DJ256 este o unitate performantă pe 16 biți, destinată în special aplicațiilor automotive și industriale, având următoarele caracteristici principale:

- Unitate centrala HCS12
- Include interfaţă pentru magistrală externă multiplexată, modul de breackpoint, modul pentru depanare;
- Include modul de tact complex: oscilator cu consum mic, PLL, monitor de tact, COP watchdog;
- Porturi pe 8 şi 4 biţi, cu facilităţi de întrerupere;
- Contor pe 16 biţi cu 8 canale de captare/ comparare; contor acumulator (2 pe 8 biţi sau 1 pe 16 biţi);
- 8 canale PWM;
- 2 interfeţe seriale asincrone şi 3 interfeţe seriale sincrone;
- Interfaţă serială pentru aplicaţii automotive şi interfaţă Inter IC (IIC);

69. Prezentați 3 caracteristici ale familiei PIC 16F8X neîntâlnite la celelalte 2 microcontrolere.

Scurt:

Caracteristici neintalnite la celelalte microcontrolere:

Arhitectura Harvard (8051 si Motorola folosesc Von Neumann). Exista 2 magistrale de date interne, pentru cod şi pentru date;

Implementeaza arhitectura RISC. Are un set de doar 35 de instructiuni spre deosebire de celelalte care au mult mai multe.

## Formatul Fix al Instrucțiunilor (14 biți)

O particularitate unică a acestei familii (Mid-range PIC) este lățimea cuvântului de instrucțiune, care este diferită de lățimea datelor.

- Caracteristică: Toate cele 35 de instrucțiuni au o lungime fixă de 14 biți (single word).

Comparație: La 8051 și Motorola, instrucțiunile au lungimi variabile (1, 2, 3 sau mai mulți octeți), ceea ce complică decodificarea. La PIC, magistrala de program fiind de 14 biți, instrucțiunea este adusă și executată "dintr-o bucată", asigurând faptul că majoritatea instrucțiunilor se execută într-un singur ciclu mașină

Lung:

Comparând PIC 16F8X cu microcontrolerele 8051 și Motorola (MC9S12), se disting următoarele trei caracteristici unice:

### 1. Arhitectură Harvard Pură (cu lățimi de bandă diferite)

Spre deosebire de 8051 și Motorola, care folosesc arhitectura Von Neumann (sau variații ale acesteia), PIC implementează o arhitectură Harvard fizic distinctă:

- **Magistrale separate:** Există două magistrale de date interne separate: una pentru memoria de program (cod) și una pentru memoria de date.
- **Lățime diferită:** Memoria de program este organizată pe **14 biți**, în timp ce memoria de date este pe **8 biți**. Acest lucru permite transferul simultan al instrucțiunilor și datelor, eliminând "gâtuirea" specifică arhitecturii Von Neumann și crescând viteza.

### 2. Arhitectură RISC cu Format Fix al Instrucțiunilor

Aceasta este o diferență majoră față de celelalte două familii (care sunt de tip CISC):

- **Set redus:** PIC are un set de doar **35 de instrucțiuni**.
- **Format Fix (14 biți):** Toate instrucțiunile au o lungime fixă de un singur cuvânt (14 biți).
  - *Comparație:* La 8051 și Motorola, instrucțiunile au lungimi variabile (1, 2, 3 octeți), ceea ce complică decodificarea. Formatul fix al PIC permite ca instrucțiunea să fie adusă și executată "dintr-o bucată", asigurând execuția majorității instrucțiunilor într-un singur ciclu mașină.

### 3. Imposibilitatea accesării memoriei externe (Arhitectură închisă)

- PIC 16F8X este un sistem "închis": **nu poate accesa memorie externă** de program sau de date.
- Nu dispune de pini pentru magistrale de adrese și date externe.
  - *Avantaj:* Aceasta duce la eliberarea pinilor pentru funcții de Intrare/Ieșire (I/E) și la dimensiuni reduse ale circuitului.
  - *Comparație:* Atât 8051, cât și Motorola HCS12 sunt proiectate cu magistrale externe multiplexate pentru extinderea memoriei.

> **Notă suplimentară:** O altă caracteristică hardware distinctivă este curentul mare generat la pinii de ieșire (**20-25mA**), permițând conectarea directă a LED-urilor, spre deosebire de celelalte familii care necesită drivere.

70. Justificați impactul frecvenței și al tensiunii de alimentare asupra consumului sistemelor încorporate. Soluții.

## 1. Justificare (Analiza Consumului de Putere)

Consumul de putere într-un circuit CMOS este determinat de formula puterii totale:

$$P_{total} = P_{static} + P_{short} + C_{SW} \cdot f \cdot V_{dd}^2 + P_{glitching}$$

**Explicarea termenilor:**

- $P_{static}$ **(Puterea statică):** Este puterea consumată datorită curenților reziduali (de scurgere). Aceasta are o valoare nenulă chiar și dacă circuitul nu lucrează (nu comută).
  - *Notă:* Pentru circuitele CMOS clasice, are o pondere neînsemnată, dar devine importantă la tehnologiile foarte mici.
- **Componentele Dinamice (următoarele 3):**
  - $P_{short}$**:** Apare la tranziții, când cele două tranzistoare din etajul final sunt simultan deschise pentru o perioadă foarte scurtă (scurtcircuit). Prin proiectare corectă, se menține la < 10% din puterea dinamică.
  - $C_{SW} \cdot f \cdot V_{dd}^2$**:** Este componenta capacitivă (de încărcare/descărcare a capacităților parazite). Este **componenta dominantă** a consumului.
  - $P_{glitching}$**:** Consum cauzat de tranziții inutile/false ale semnalelor.

**Impactul Frecvenței și Tensiunii:**

Analizând componenta dominantă ($C_{SW} \cdot f \cdot V_{dd}^2$), observăm:

- **Impactul Frecvenței (**$f$**):** Consumul crește **liniar** cu frecvența.
- **Impactul Tensiunii (**$V_{dd}$**):** Consumul crește **pătratic** cu tensiunea.
  - *Concluzie:* Reducerea tensiunii ($V_{dd}$) are cel mai mare impact asupra economisirii energiei, deoarece reducerea ei scade puterea "la pătrat".

## 2. Soluții pentru reducerea consumului

**A. În faza de Proiectare:**

1. **Deconectarea internă dinamică (Clock Gating):** Oprirea tactului pentru blocurile neutilizate.
2. **Codificarea pe magistrale:** Utilizarea codurilor **Gray**, **Bus-Invert** sau **T0** pentru a minimiza numărul de tranziții (comutări) pe magistralele externe, reducând $C_{SW}$ (capacitatea comutată).
3. **Codificarea instrucțiunilor:** Minimizarea tranzițiilor în procesul de aducere a instrucțiunilor (Fetch).

**B. În faza de Funcționare:**

1. **Modificarea dinamică a frecvenței și tensiunii (DVFS):** Adaptarea resurselor la cerințele task-ului curent. De exemplu, scăderea frecvenței prin PLL permite și scăderea tensiunii de alimentare, reducând drastic consumul ($P \sim V^2$).

2. **Utilizarea modurilor "Low Power":**
   - *Idle/Sleep:* CPU oprit, periferice active.
   - *Stop:* Tot circuitul oprit (inclusiv oscilatorul), consum minim ($P \approx P_{static}$).
3. **Optimizare Software:** Utilizarea paginii 0 pentru variabile, evitarea calculelor în virgulă mobilă inutile.